Projeto da Disciplina: Visão Computacional

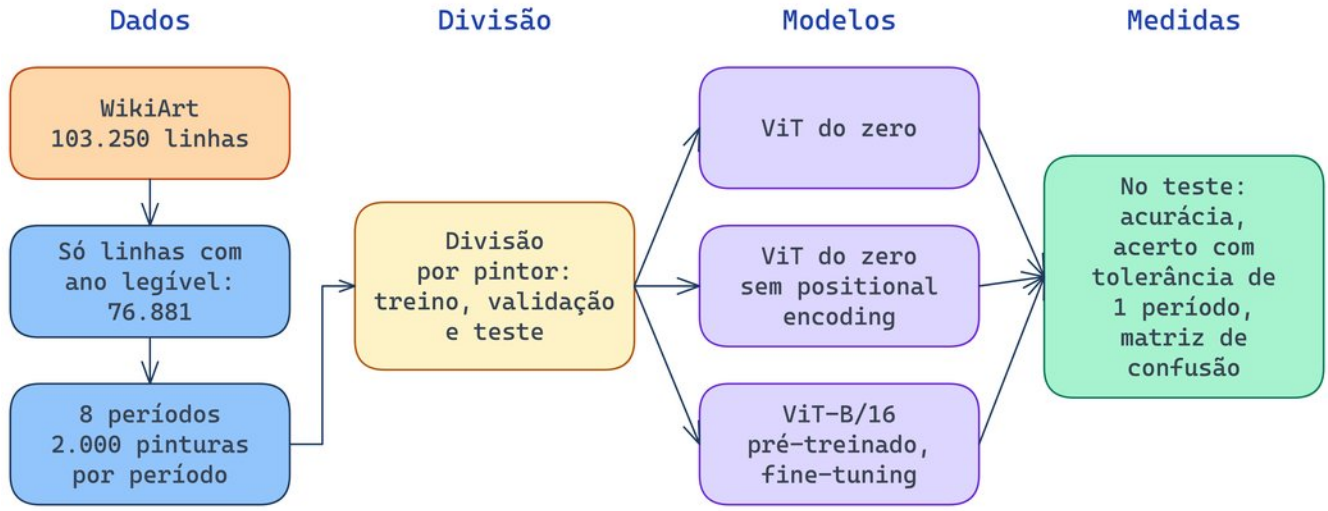
- **Aluno:** Anderson Corrêa
- **Curso:** Deep Learning and Vision
- **Disciplina:** Computer Vision (INFNET 26E3_3)

Como ler este relatório

- Cada atividade tem uma seção com: problema, decisões técnicas, resultados e análise crítica.
- Os números e gráficos vêm da execução dos notebooks no Google Colab com GPU T4.
- Cada notebook roda do início ao fim sem alterações e baixa os próprios dados.
- Repositório do projeto: github.com/andersonfpcorrea/26E3_3
- Os links abaixo abrem cada notebook direto no Colab. Depois de abrir, escolha a GPU T4 em Ambiente de execução e use Executar tudo.
  - Atividade 1: A1_vision_transformers.ipynb
  - Atividade 2: A2_clip_ads16.ipynb
  - Atividade 3: A3_cnn_kaggle.ipynb
  - Atividade 4.1: A4_estudo_caso_raio_x.ipynb
- O arquivo de entrega também traz os módulos-fonte e os testes automáticos (`src/dlv/` e `tests/`), com os scripts e as células-fonte que os testes usam.
- Para rodar os testes a partir do arquivo de entrega: `make setup`, depois `make test`.

Atividade 1: Datação de pinturas com Vision Transformers

Problema



Atividade 1: dos dados às medidas

- Tarefa: dizer em qual de 8 períodos uma pintura foi feita, olhando só a imagem.
- Dataset: WikiArt, na cópia `Artificio/WikiArt` do Hugging Face. São 103.250 linhas, com imagens já em 256x256.
- Origem e licença: é um upload de terceiros, sem licença declarada. O uso aqui é só para estudo.
- O ano vem da coluna `date`, que é texto em formatos variados (`1920`, `1997.0`, `c.1904`, `None`). 76.881 linhas têm um ano que pode ser lido, 74,5% do total. As outras foram descartadas.
- Três modelos são comparados:
  - um Vision Transformer construído à mão e treinado do zero;
  - o mesmo modelo sem positional encoding;
  - um ViT-B/16 pré-treinado na ImageNet, com fine-tuning.

Os oito períodos

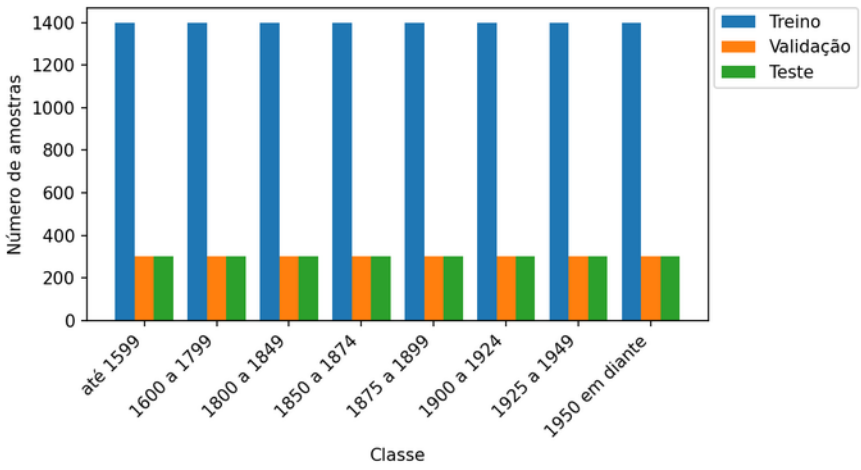
Período	Pinturas com ano no dataset	Pinturas na amostra
até 1599	5.427	2.000
1600 a 1799	6.686	2.000
1800 a 1849	4.467	2.000
1850 a 1874	5.453	2.000
1875 a 1899	13.640	2.000
1900 a 1924	17.235	2.000
1925 a 1949	9.927	2.000
1950 em diante	14.046	2.000
Total	76.881	16.000

- Os períodos antigos são mais largos porque têm menos pinturas. O primeiro cobre séculos, o segundo 200 anos e o terceiro 50. Os quatro seguintes cobrem 25 anos cada.
- A amostra é balanceada: 2.000 pinturas por período, sorteadas com seed 42. São 16.000 pinturas, 20,8% das que têm ano.
- Com 8 classes do mesmo tamanho, um chute ao acaso acerta 12,5%.

Divisão por pintor

Conjunto	Pinturas	Por período	Pintores
Treino	11.200	1.400	1.107
Validação	2.400	300	223
Teste	2.400	300	223

- Todas as pinturas de um pintor ficam no mesmo conjunto. A amostra tem 1.553 pintores, e nenhum aparece em dois conjuntos.
- Motivo: um pintor trabalha poucas décadas e tem um jeito próprio. Se o mesmo pintor estivesse no treino e no teste, o modelo poderia acertar o período só por reconhecer o pintor. A divisão por pintor mede o que interessa: datar a obra de um pintor que o modelo nunca viu.
- Nenhuma pintura da amostra está sem pintor conhecido. Nenhum pintor da amostra aparece com duas grafias.
- Os conjuntos ficaram balanceados por período, como mostra a figura.
- Um pintor muito produtivo pesa no teste de dois períodos: Max Ernst tem 56 das 300 pinturas de teste de 1925 a 1949 (18,7%), e Sam Francis tem 46 das 300 de 1950 em diante (15,3%).



Número de pinturas por período em cada conjunto de dados

Estilo e data são coisas diferentes

- O rótulo é o ano da pintura. O que a imagem mostra é o estilo.
- Um pintor pode trabalhar em um estilo mais antigo que o seu tempo. Nesses casos a imagem aponta para um período e o rótulo para outro.
- Os anos vêm do upload do WikiArt e não foram conferidos. Datas aproximadas, como `c.1904`, entram como se fossem exatas.
- Por isso existe um teto de acerto abaixo de 100%, que este trabalho não mediu.

Decisões técnicas

Attention e encoder construídos do zero

Tudo está em `src/dlv/attention.py`, escrito com camadas lineares e softmax. Os módulos prontos de attention do PyTorch (`nn.MultiheadAttention`, `nn.TransformerEncoderLayer`) não são usados.

- **Scaled dot-product attention: função `scaled_dot_product_attention`.**
  - Calcula `softmax(Q K^T / sqrt(d)) V` e devolve a saída e os pesos.
  - A divisão por `sqrt(d)` evita que os produtos internos cresçam com a dimensão e saturem o softmax.
  - Aceita uma máscara opcional.
- **Multi-head attention: módulo `MultiHeadAttention`.**
  - É o módulo que usa a função acima, uma vez por head.
  - Cada head tem as suas próprias projeções de query, key e value: três camadas lineares de 192 para 64 dimensões.
  - As saídas das 3 heads são concatenadas (3 x 64 = 192) e passam por uma projeção final de 192 para 192.
  - O módulo devolve também os pesos de cada head, no formato (lote, heads, tokens, tokens).
- **Encoder: módulo `TransformerEncoderBlock`.**
  - LayerNorm, multi-head attention e soma com a entrada (residual connection).
  - LayerNorm, feedforward de duas camadas (192 para 768, GELU, 768 para 192) e outra soma com a entrada.
  - A LayerNorm vem antes de cada parte (pre-norm), como no ViT original.

Como é testado (`tests/test_attention.py`, 13 testes em CPU)

- Os arquivos de teste estão no arquivo de entrega, na pasta `tests/`.
- A saída da função é igual à fórmula calculada à mão.
- Os pesos são não negativos e cada linha soma 1.
- Uma posição mascarada recebe peso zero. Uma linha toda mascarada não gera NaN, nem na saída nem no gradiente.
- Keys iguais dão pesos uniformes.
- O módulo tem 3 x 3 + 1 = 10 camadas lineares: uma por head e por papel, mais a projeção final.
- Mudar as projeções de uma head muda só os pesos dessa head. As outras duas ficam idênticas. Isso prova que as projeções são independentes.
- Com as últimas projeções zeradas, o bloco devolve a própria entrada. Isso prova que as residual connections existem.
- O gradiente chega a todos os parâmetros do bloco, menos aos vieses das projeções de key. O gradiente deles é zero por construção, porque o softmax ignora uma constante somada à linha.

Vision Transformer

O modelo está em `src/dlv/vit.py`, classe `VisionTransformer`. O caminho da imagem até os logits:

1. **Patch embedding** (`PatchEmbedding`): a imagem de 64x64 é cortada em patches de 8x8 sem sobreposição. São 8 x 8 = 64 patches. Cada patch vira um vetor de 192 dimensões por uma projeção linear, feita com uma convolução de kernel 8 e passo 8.
2. **CLS token**: um vetor de 192 dimensões, aprendido no treino, é colocado no início da sequência. A sequência passa a ter 64 + 1 = 65 tokens.
3. **Positional encoding**: um vetor aprendido por posição, somado a cada token. São 65 x 192 = 12.480 parâmetros.
4. **Encoder**: 6 blocos `TransformerEncoderBlock`, com 3 heads cada e dropout de 0,1.

5. **Saída**: LayerNorm final, depois uma camada linear de 192 para 8 aplicada só ao CLS token. O resultado são 8 logits, um por período.

- Total: 2.720.840 parâmetros.
- O modelo também devolve os pesos de attention de cada bloco. A função `class_token_attention` pega a linha do CLS token e a organiza na grade de 8x8 patches.

Como é testado (`tests/test_vit.py`, 14 testes em CPU)

- Os arquivos de teste estão no arquivo de entrega, na pasta `tests/`.
- Uma imagem de 64x64 vira 64 tokens de 192 dimensões. Cada token depende só do seu patch.
- O modelo devolve 8 logits por imagem.
- O CLS token e o positional encoding são parâmetros treináveis, e o positional encoding tem 65 posições.
- O mapa de attention lê a linha do CLS token, tem o formato da grade e soma no máximo 1.
- No modelo pré-treinado, a classification head é trocada e todas as camadas continuam treináveis.

Por que attention precisa de positional encoding

O argumento

- A attention compara cada token com todos os outros por produto interno. Nenhuma parte da conta usa a posição do token na sequência.
- Se os tokens de entrada trocam de lugar, as saídas trocam de lugar do mesmo jeito, com os mesmos valores. A attention trata a entrada como um conjunto, sem ordem.
- A feedforward e a LayerNorm agem em cada token sozinho, então também não enxergam ordem.
- No ViT, o CLS token soma informação de todos os patches. Sem posição, o que ele recebe é igual para qualquer arrumação dos patches.
- O positional encoding resolve isso: soma a cada token um vetor que depende da posição. Dois patches iguais em lugares diferentes passam a ser tokens diferentes.

O teste que prova o mecanismo

- `test_without_positional_embedding_patch_order_does_not_matter`: no modelo sem positional encoding, embaralhar os 64 patches deixa os logits iguais.
- `test_with_positional_embedding_patch_order_matters`: no modelo com positional encoding, o mesmo embaralhamento muda os logits.

O experimento

- O modelo foi treinado uma segunda vez sem positional encoding. Mesma seed, mesmos dados, mesmo treino.
- Ele tem 2.708.360 parâmetros, 12.480 a menos que o modelo completo: os do positional encoding.
- Resultado no teste: 780 acertos em 2.400 com posição, 779 sem posição. Diferença de 1 pintura.
- Os dois modelos são diferentes de fato: as matrizes de confusão não são iguais. Em 1925 a 1949, por exemplo, um acerta 48 e o outro 65.
- Com 2.400 pinturas e acurácia perto de 32%, o erro padrão é de cerca de 1 ponto percentual: raiz de (0,325 x 0,675 / 2.400) = 0,0096. A diferença medida, de 0,04 ponto, é muito menor que isso. E há uma única seed por modelo.
- Leitura: nesta tarefa, a 64x64 e com este modelo pequeno, a informação de posição não ajudou de forma mensurável. Outras tarefas e resoluções não foram testadas.

Por que isso pode acontecer aqui

- Hipótese: as pistas de período que sobrevivem a 64x64 são em boa parte globais, como paleta de cores, contraste e textura da pincelada. Um conjunto de patches sem ordem já carrega essas pistas.

Hiperparâmetros

ViT treinado do zero (com e sem posição)

- **Entrada de 64x64 e patch de 8x8.** Dá 65 tokens, o que deixa o treino rápido na T4: cerca de 8 segundos por época. As imagens reduzidas cabem na memória da GPU.
- **192 dimensões, 6 blocos, 3 heads.** Um modelo pequeno, de 2,7 milhões de parâmetros, para 11.200 imagens de treino. São 243 parâmetros por imagem. Um modelo maior tenderia a ter ainda mais overfitting.
- **AdamW, weight decay 0,05.** É o otimizador usual para Transformers. O weight decay é a regularização principal.
- **Taxa de aprendizado 0,001, com 5 épocas de subida linear e depois queda em cosseno.** Transformers treinados do zero são instáveis no começo. A subida gradual evita passos grandes enquanto os pesos ainda são aleatórios.
- **Lotes de 256 imagens.** São 44 passos por época.
- **Augmentation: recorte aleatório com margem de 4 pixels e flip horizontal.** São leves e não mudam as cores, que são pista de período.
- **Normalização com a média e o desvio padrão de cada canal, medidos no treino.**
- **Dropout 0,1. Perda: entropia cruzada. Seed 42.**
- **40 épocas.** A primeira execução usou 60 épocas e levou 27,7 minutos no total, perto do limite de 30 minutos do notebook. O corte para 40 foi por custo. Nessa primeira execução, a acurácia de validação era 32,9% na época 40 e 32,0% na época 60. Na execução final, a melhor acurácia de validação (32,2%) aparece nas épocas 27 e 37, e a época 40 fica em 31,4%. As duas execuções usam agendas de taxa de aprendizado diferentes, então a comparação é indicativa.

ViT-B/16 pré-treinado

- **Pesos `IMAGENET1K_V1` do torchvision.** O modelo tem 85.804.808 parâmetros.
- **Classification head trocada.** A camada final, de 1.000 saídas, deu lugar a uma camada linear nova de 8 saídas.
- **Fine-tuning de todas as camadas.** Pinturas são um domínio distante das fotos da ImageNet, e há 11.200 imagens de treino. Congelar o backbone, como na Atividade 3, poderia deixar as features presas às fotos.
- **Duas taxas de aprendizado: 0,00003 no backbone e 0,001 na camada nova.** A camada nova começa aleatória e precisa de passos grandes. O backbone já está treinado, e passos grandes apagariam o que ele sabe.
- **Entrada igual à do pré-treinamento:** redimensionar para 256, recorte central de 224x224, média e desvio padrão da ImageNet.
- **Lotes de 32 imagens e precisão mista.** Mantêm o pico de memória da GPU em 3.971,6 MB.
- **Sem augmentation e com taxa de aprendizado constante.** Foi a configuração mais simples. É um limite: ver a análise crítica.
- **2 épocas.** A primeira execução usou 3, com acurácia de validação de 49,2% na época 2 e 47,8% na época 3. O corte foi por custo. Na execução final, a perda de validação já sobe da época 1 para a 2, como mostram as curvas de treino.

Custo da execução final

- Notebook inteiro: 963,4 segundos (16,1 minutos) na T4.
- Pico de memória: 3.971,6 MB na GPU e 3.033,4 MB de RAM.

Resultados

Comparação dos três modelos no teste (2.400 pinturas)

Modelo	Acurácia	Acertos	Acerto com tolerância de 1 período	F1 macro	Distância média	Parâmetros	Treino
ViT do zero	32,50%	780	64,0%	0,320	1,49	2.720.840	304,6 s
ViT do zero sem posição	32,46%	779	64,5%	0,322	1,47	2.708.360	318,3 s
ViT-B/16 pré-treinado	47,08%	1.130	80,5%	0,455	0,93	85.804.808	267,4 s
Chute ao acaso	12,50%	300	34,4%		2,63		

- **Acerto com tolerância de 1 período:** conta como acerto o período exato ou um vizinho.
- **Distância média:** quantos períodos, em média, separam a previsão do período real.
- **Linha do chute ao acaso:** valores calculados por conta, sem execução.
  - Os 2 períodos das pontas têm 2 respostas aceitas e os outros 6 têm 3: (2 x 2 + 6 x 3) / 64 = 34,4%.
  - A distância média entre dois períodos sorteados é 168 / 64 = 2,63.
- O modelo pré-treinado acerta 1.130 - 780 = 350 pinturas a mais que o ViT do zero: 14,6 pontos percentuais.
- Os dois modelos do zero ficam cerca de 20 pontos acima do acaso. Eles aprendem algo, mas pouco.

Acurácia por período no teste (300 pinturas por período)

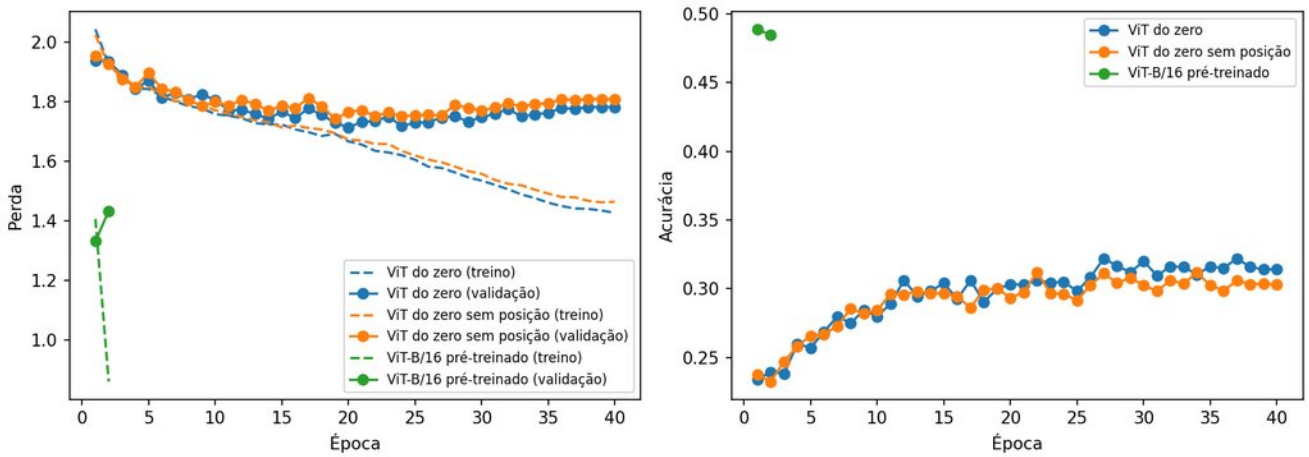
Período	ViT do zero	Sem posição	ViT-B/16 pré-treinado
até 1599	51,0%	51,3%	89,7%
1600 a 1799	45,7%	45,0%	46,0%
1800 a 1849	25,7%	24,3%	41,3%
1850 a 1874	24,7%	24,0%	42,0%
1875 a 1899	28,0%	23,0%	9,7%
1900 a 1924	17,3%	17,7%	32,3%
1925 a 1949	16,0%	21,7%	55,0%
1950 em diante	51,7%	52,7%	60,7%

- Os três modelos acertam mais nas pontas. São os períodos mais diferentes dos outros, e o primeiro é o mais largo.

Curvas de treino

- **Os dois modelos do zero têm overfitting.**
  - A acurácia de treino sobe até o fim: 46,6% e 45,2% na época 40.
  - A acurácia de validação para perto de 31%.
  - A perda de validação do ViT do zero tem o mínimo na época 20 (1,713) e sobe até 1,781. A de treino continua caindo.

    - Diferença final entre treino e validação no ViT do zero: 46,56% - 31,42% = 15,1 pontos.
- **As curvas com e sem posição andam juntas** do início ao fim. A diferença entre elas é do tamanho da oscilação de uma época para outra. A diferença final na validação, 31,4% com posição e 30,3% sem, está dentro do ruído.
- **O modelo pré-treinado também tem overfitting, e mais rápido.** Na época 2 o treino chega a 68,0% e a validação fica em 48,5%. A perda de validação sobe de 1,332 para 1,433.
- Os pesos avaliados no teste são os da última época, nos três modelos.

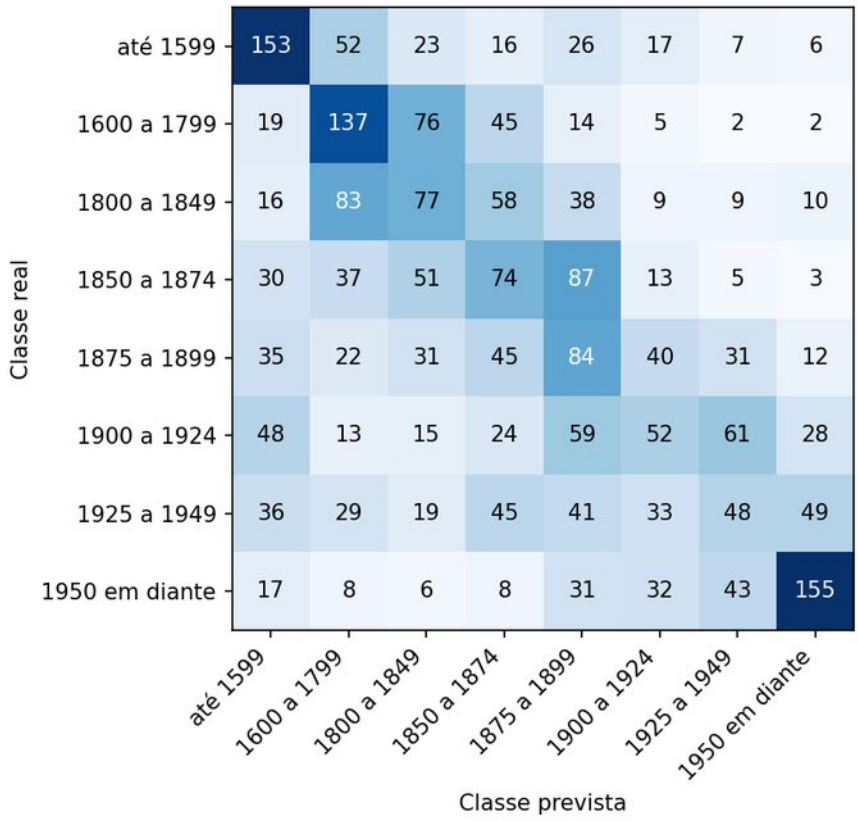


Curvas de perda e acurácia por época dos três modelos

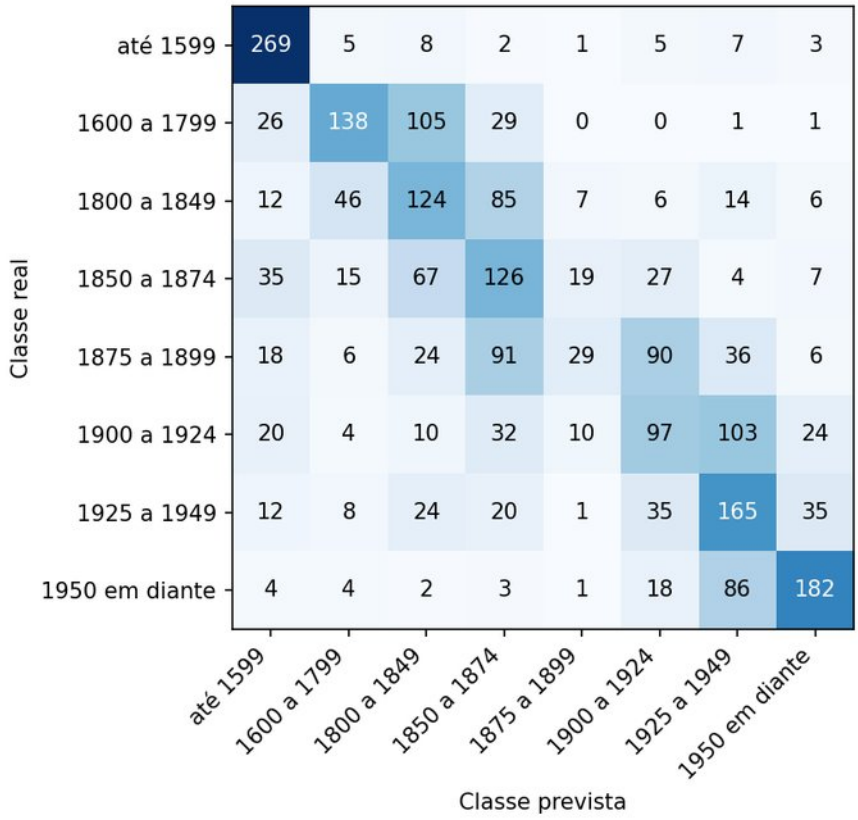
Onde caem os erros

Modelo	Período exato	Vizinho (1 período)	Longe (2 ou mais)
ViT do zero	780 (32,5%)	756 (31,5%)	864 (36,0%)
ViT-B/16 pré-treinado	1.130 (47,1%)	803 (33,5%)	467 (19,5%)

- **ViT do zero:** os erros se espalham pela matriz. Mais de um terço das previsões cai a 2 períodos ou mais do real.
    - Há erros de séculos: 48 pinturas de 1900 a 1924 e 36 de 1925 a 1949 foram datadas como "até 1599".
    - O miolo é confuso: de 1800 a 1949, nenhum período passa de 84 acertos em 300.
- **ViT-B/16 pré-treinado:** os erros ficam junto da diagonal.
    - 1600 a 1799 perde 105 pinturas para 1800 a 1849.
    - 1900 a 1924 perde 103 para 1925 a 1949, mais do que acerta (97).
    - 1950 em diante perde 86 para 1925 a 1949.
    - "até 1599" é quase resolvido: 269 de 300.
- **O buraco em 1875 a 1899.** Das 300 pinturas desse período, o modelo pré-treinado acerta 29 (9,7%, abaixo do acaso e pior que os 28,0% do ViT do zero), manda 91 para 1850 a 1874 e 90 para 1900 a 1924. No teste inteiro, ele só responde "1875 a 1899" 68 vezes em 2.400. O modelo quase não usa essa classe e divide as pinturas dela entre os dois vizinhos. Como os erros caem nos vizinhos, a métrica de 1 período de tolerância esconde esse defeito. A acurácia por período mostra.



Matriz de confusão do ViT do zero, no conjunto de teste

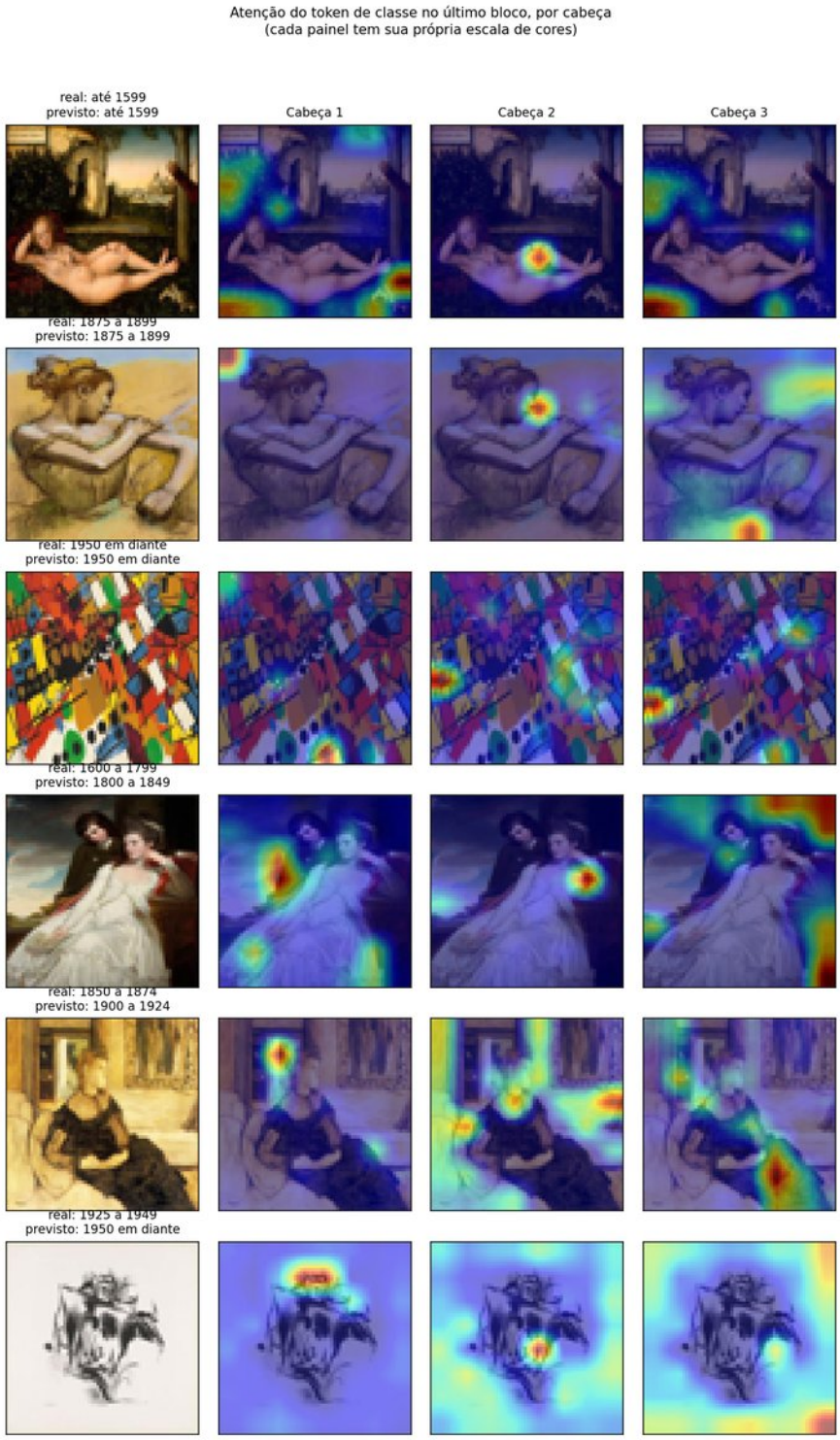


Matriz de confusão do ViT-B/16 pré-treinado, no conjunto de teste

O que o modelo observa

O que as figuras mostram

- Modelo: o ViT treinado do zero, com positional encoding.
- Seis pinturas do teste: três que o modelo acertou e três que errou, de períodos diferentes. A escolha é fixa (a primeira pintura de cada período escolhido), sem seleção à mão.
- Primeira figura: attention do CLS token para cada um dos 64 patches, em cada uma das 3 heads do último bloco. É a attention que alimenta a classificação.
- Segunda figura: a mesma attention no primeiro bloco, na média das 3 heads.
- Cada painel tem a sua própria escala de cores. O vermelho marca o maior valor daquele painel. As cores não servem para comparar um painel com outro.
- A grade é de 8x8 patches, ampliada sobre a imagem. Cada mancha cobre uma região de pelo menos 8x8 pixels.



Attention do CLS token no último bloco, uma coluna por head. Cada painel tem sua própria escala de cores

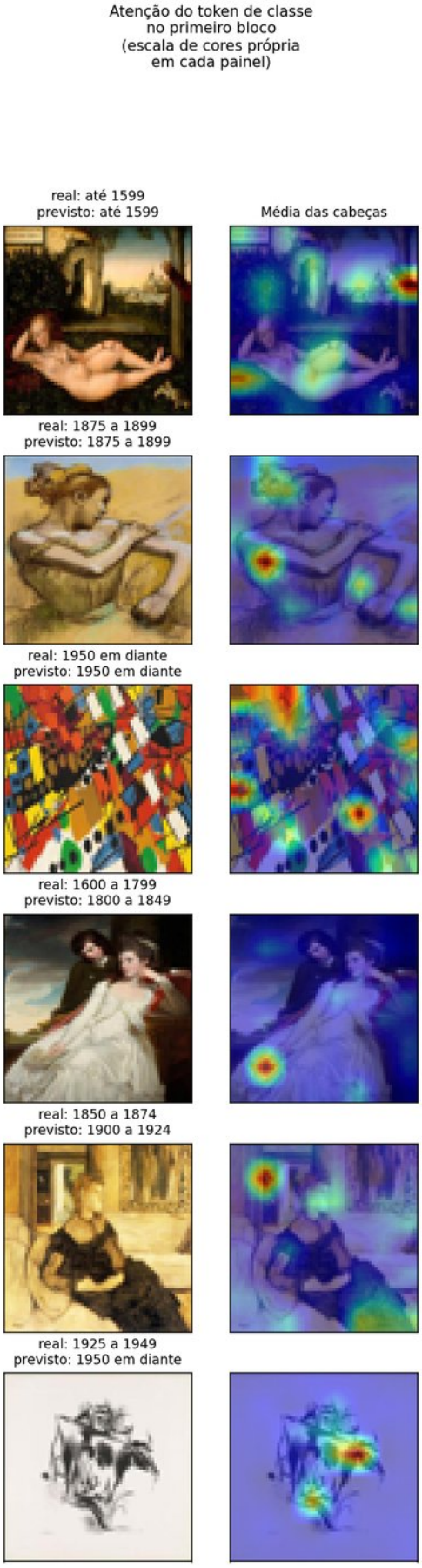
Último bloco, pintura por pintura

Pintura	Head 1	Head 2	Head 3
1. The Nymph of the Spring, Lucas Cranach, 1540 (acerto)	Cantos inferiores, mais forte no direito, com pontos fracos no canto superior direito e na borda esquerda	Um único ponto forte, sobre o quadril da figura deitada	Canto inferior esquerdo e borda esquerda
2. Torso of a Dancer, Edgar Degas, 1899 (acerto)	Só o canto superior esquerdo, no fundo liso	Um ponto ao lado do rosto, na altura do queixo e do ombro	Base da imagem, no centro, e faixa difusa no alto à direita
3. Pastorale, Ernst Wilhelm Nay, 1954 (acerto)	Base da imagem, sobre um disco verde	Borda esquerda, com faixa fraca que atravessa o centro	Borda esquerda, mais abaixo, e pontos menores à direita
4. Jane Maxwell com o filho, George Romney, 1778 (erro: previsto 1800 a 1849)	Ponto forte na área escura à esquerda da mulher, junto ao casaco do rapaz, e pontos mais fracos no vestido	Um ponto no braço erguido da mulher, à direita	Canto superior direito e borda direita, no fundo
5. Madame Gobillard, Edgar Degas, 1869 (erro: previsto 1900 a 1924)	Um ponto acima e à esquerda da cabeça, no fundo	Espalhada pela parede e pelas bordas, com um ponto na altura do pescoço da figura	Parte de baixo do vestido, à direita
6. Milking the Cow, Yasuo Kuniyoshi, 1927 (erro: previsto 1950 em diante)	Topo do desenho	Centro inferior do desenho, e difusa no papel branco	Difusa no papel branco, mais forte na borda direita e no canto inferior direito

A pintura 3 é abstrata. Não há figura para a attention seguir.

Regiões que emergem

- **Head 2 cai mais vezes sobre a figura**, nas pinturas 1, 2, 4, 5 e 6. É um ponto pequeno, de um ou dois patches, e muda de parte do corpo a cada pintura.
- **Heads 1 e 3 caem mais vezes em cantos, bordas e fundo.**
- **O padrão não é estável.** Na pintura 3 a head 2 está na borda, e na 5 ela é a mais espalhada das três. A head 1 cai sobre a cena nas pinturas 4 e 6. Seis pinturas é pouco para afirmar um papel fixo por head.
- **Não há diferença visível entre os mapas dos acertos e os dos erros.**



Attention do CLS token no primeiro bloco, média das heads. Cada painel tem sua própria escala de cores

Primeiro bloco

- Os mapas são manchas locais. Parecem seguir pontos de contraste forte:
  - pintura 1: borda direita no alto, ao lado do céu claro, com um ponto mais fraco no céu;
  - pintura 2: braço da figura, à esquerda;
  - pintura 3: vários pontos entre as formas coloridas;
  - pintura 4: parte de baixo do vestido branco, à esquerda;
  - pintura 5: parede no alto à esquerda;
  - pintura 6: traços escuros do desenho, sem nada no papel branco.
- No primeiro bloco os tokens ainda são quase só o patch e a posição. A attention parece seguir cor e contraste do próprio patch.

Interpretação

- Os mapas não mostram uma história limpa, do tipo "o modelo olha para o rosto". É o esperado para um modelo pequeno, que acerta 32,5% no teste, com patches de 8x8 em uma imagem de 64x64. Mapas com contorno de objeto são relatados em ViTs treinados com muito mais dados, sobretudo com self-supervised learning.
- A attention em cantos e fundo combina com a hipótese da seção sobre positional encoding: o modelo pode estar lendo cor e textura do fundo, que são pistas globais de período. Os mapas não provam isso.
- Limite do método: o mapa mostra para onde o CLS token olha em um bloco. Ele não mostra o quanto cada região pesou na decisão final.

Erros por movimento artístico

- As previsões do teste foram agrupadas pela coluna `style` do dataset.
- Entram os 21 movimentos com pelo menos 20 pinturas no teste. Eles somam 2.102 das 2.400 pinturas.

Movimentos com mais de 50 pinturas no teste

Movimento	Pinturas	Acurácia, ViT do zero	Acurácia, pré-treinado	Distância média, pré-treinado
Romanticism	437	25,6%	35,5%	1,05
Realism	244	28,3%	34,4%	1,05
Northern Renaissance	212	50,9%	90,6%	0,38
Impressionism	200	27,5%	26,5%	1,18
Rococo	152	43,4%	46,7%	0,60
Expressionism	128	14,8%	45,3%	1,38
Surrealism	121	19,0%	52,9%	1,27
Symbolism	98	16,3%	14,3%	1,71
Post-Impressionism	75	26,7%	33,3%	1,13
Naïve Art (Primitivism)	65	29,2%	55,4%	1,05
Abstract Expressionism	61	63,9%	75,4%	0,31
Early Renaissance	54	50,0%	87,0%	0,39

Mais bem e mais mal datados pelo modelo pré-treinado

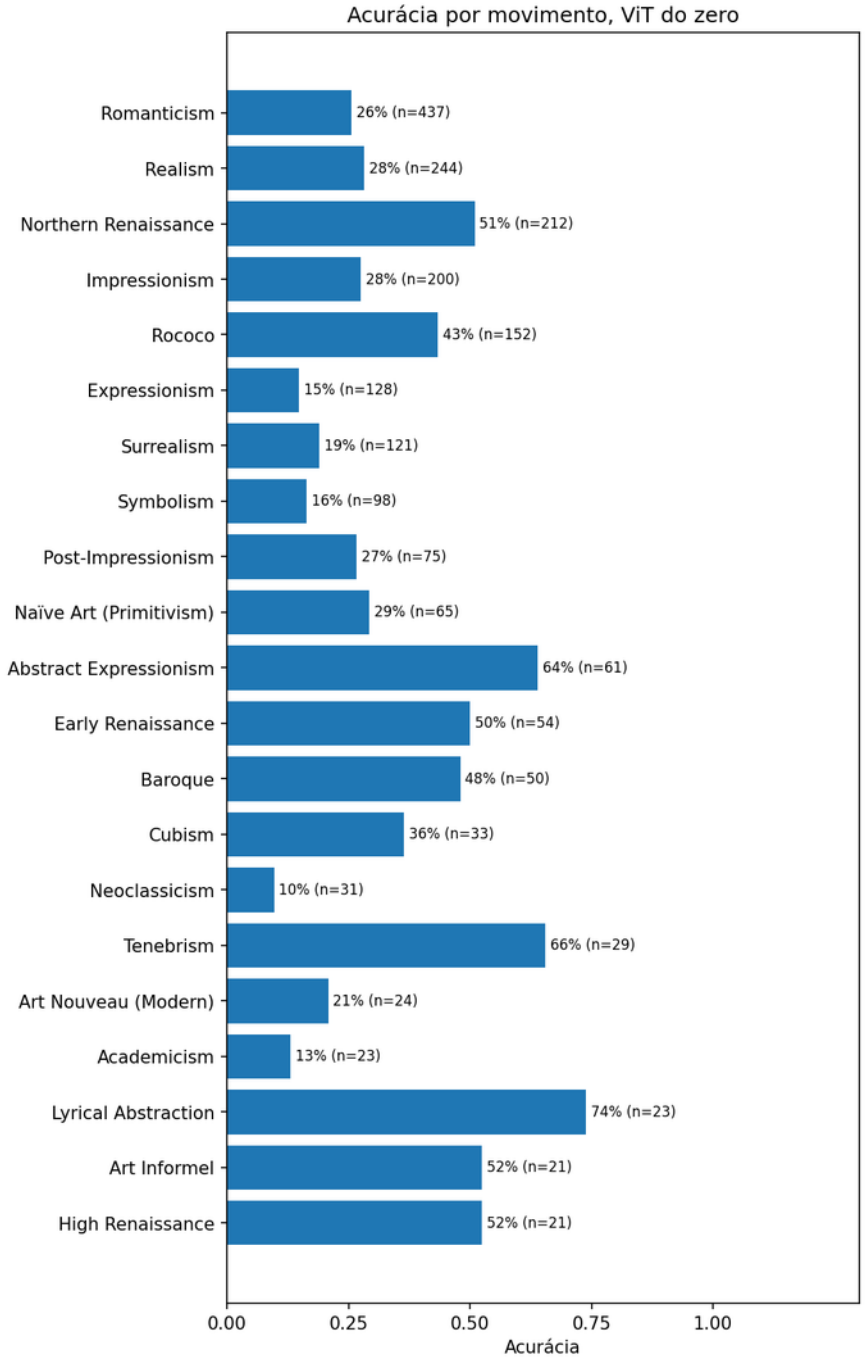
- Os mais bem datados já aparecem na tabela. Fora dela, por terem poucas pinturas, estão High Renaissance (90,5%, 19 de 21), Lyrical Abstraction (78,3%, 18 de 23) e Art Informel (81,0%, 17 de 21). O pior é Neoclassicism, com 12,9% (4 de 31). Romanticism e Realism, com 35,5% e 34,4%, somam 681 pinturas, 28% do teste.

Razão provável: quanto tempo o movimento durou em relação à largura dos períodos

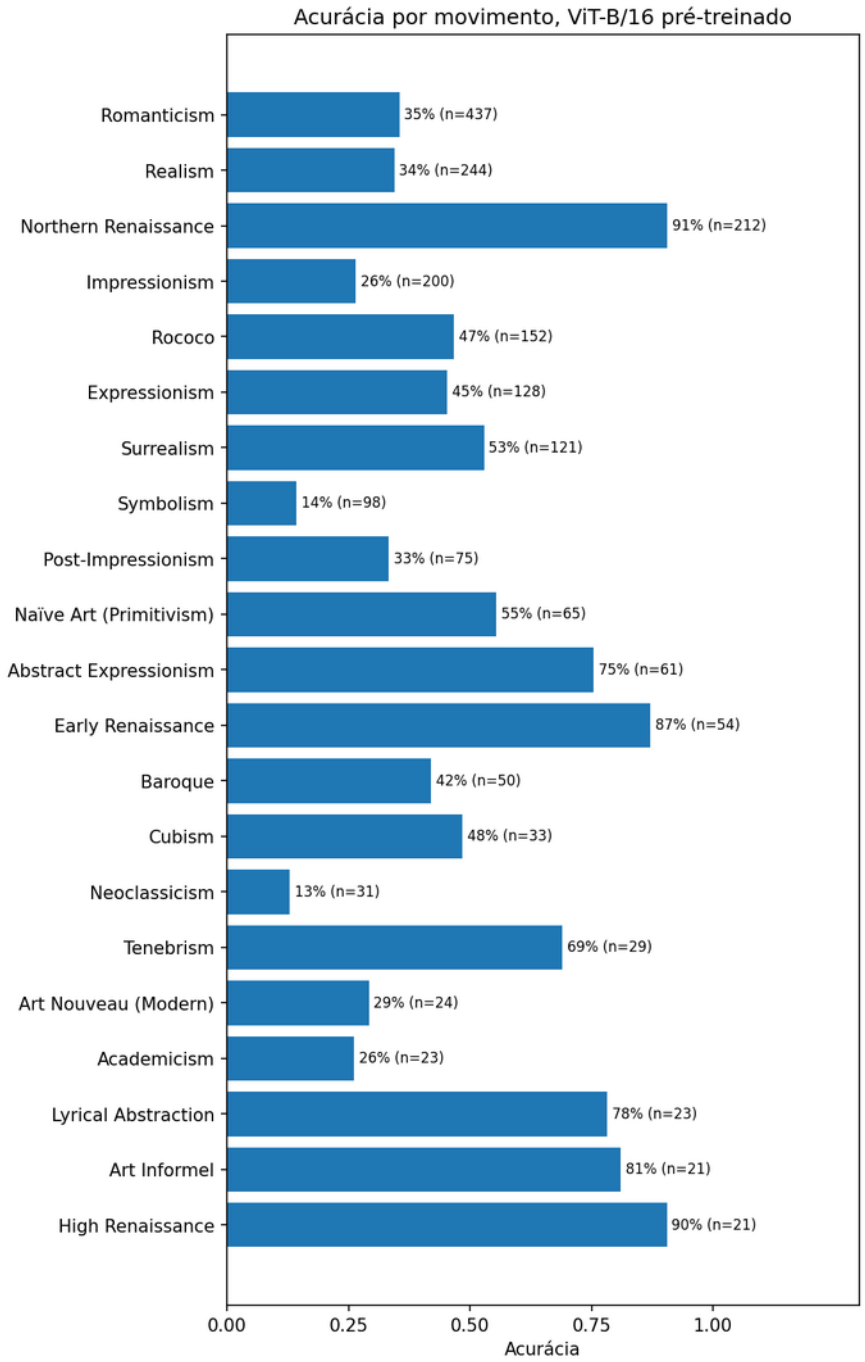
- Os movimentos do Renascimento cabem inteiros no período "até 1599", que é o mais largo. Reconhecer o movimento já dá o período certo.
- A abstração do pós-guerra cabe quase inteira em "1950 em diante", que não tem limite final.
- Romanticism, Realism, Impressionism e Symbolism são do século XIX e do começo do XX, onde os períodos têm 50 ou 25 anos. Um movimento que dura várias décadas fica partido em 2 ou 3 classes. O modelo reconhece o movimento e ainda precisa adivinhar a fatia.
- A distância média apoia essa leitura para o Impressionism: a acurácia é de 26,5%, mas o erro médio é de 1,18 período. O modelo erra por perto.
- O Neoclassicism é um caso diferente: o erro médio é de 2,03 períodos, ou seja, erra para longe. Uma razão plausível é que o estilo imita de propósito a pintura de séculos anteriores. É o exemplo mais direto de que estilo e data são coisas diferentes. São só 31 pinturas, então a medida é fraca.
- A hipótese não explica tudo: Baroque (42,0%) e Rococo (46,7%) cabem no período 1600 a 1799 e mesmo assim ficam abaixo de 50%. Os valores são do modelo pré-treinado.
- Esta explicação é uma hipótese. O notebook não guardou a distribuição de anos de cada movimento, então ela não foi conferida nos dados.

Onde o pré-treinamento ajudou mais e onde não ajudou

- Maiores ganhos, em pontos percentuais, entre os 21 movimentos: Northern Renaissance, High Renaissance (52,4% para 90,5%), Early Renaissance e Surrealism.
- Sem ganho: Impressionism, Symbolism, Baroque (48,0% para 42,0%, com 50 pinturas) e Neoclassicism (9,7% para 12,9%, uma pintura a mais em 31).
- Uma leitura possível, não conferida nos dados: Impressionism e Symbolism têm muitas pinturas do fim do século XIX, onde fica o buraco descrito em Resultados.



Acurácia por movimento artístico, ViT do zero



Acurácia por movimento artístico, ViT-B/16 pré-treinado

ViT do zero ou ViT pré-treinado

Tabela comparativa

	ViT do zero	ViT-B/16 pré-treinado
Acurácia no teste	32,5% (780 de 2.400)	47,1% (1.130 de 2.400)
Acerto com tolerância de 1 período	64,0%	80,5%
F1 macro	0,320	0,455
Distância média	1,49 período	0,93 período
Erros a 2 períodos ou mais	864	467
Parâmetros	2.720.840	85.804.808
Entrada	64x64	224x224
Épocas	40	2
Tempo de treino na T4	304,6 s	267,4 s
Tempo de avaliação do teste	0,6 s	8,7 s
Dados vistos antes	nenhum	ImageNet-1k

Escolha para este domínio: o ViT pré-treinado

- Acerta 14,6 pontos percentuais a mais no teste. O erro padrão de cada acurácia é de cerca de 1 ponto, então a diferença está muito acima do ruído.
- Custou menos tempo de treino, mesmo com 31,5 vezes mais parâmetros (85.804.808 / 2.720.840). O ViT do zero não tem dados suficientes: com 11.200 pinturas, tem o overfitting descrito em Resultados.

O que pesa contra

- O modelo é grande: avaliar o teste leva 8,7 s contra 0,6 s, e o pico de memória da GPU, de 3.971,6 MB, provavelmente vem dele.
- Ele tem um defeito que o ViT do zero não tem: quase ignora o período 1875 a 1899 (9,7% contra 28,0%).
- Os dois modelos diferem em três coisas ao mesmo tempo: pré-treinamento, tamanho e resolução de entrada (224x224 contra 64x64). O experimento não separa quanto do ganho vem de cada uma.

Para que o resultado serve

- Serve para uma primeira triagem: sugerir uma faixa de época para uma pintura sem data, a ser conferida por uma pessoa. Para datar sem conferência humana, o erro é alto demais.

Pré-treinamento: BERT e ViT

	BERT	ViT original
Dados	Texto sem rótulo	Imagens com rótulo
Tipo	Self-supervised	Supervisionado
Tarefa	Masked language modeling e next sentence prediction	Classificação da imagem
De onde vem o alvo	Do próprio texto	De rótulos dados por pessoas
O que é maximizado	A probabilidade das palavras escondidas, dado o contexto	A probabilidade da classe correta da imagem

BERT

- **Masked language modeling:** parte dos tokens da frase é escondida. O modelo precisa prever cada token escondido usando as palavras à esquerda e à direita.
- **Next sentence prediction:** o modelo recebe duas frases e diz se a segunda vem depois da primeira no texto original.
- Para ir bem nas duas tarefas, cada token precisa de uma representação que use o contexto dos dois lados.

ViT

- O artigo usa conjuntos rotulados grandes, como ImageNet-21k e JFT-300M. Ele mostra que, só com a ImageNet-1k, o ViT fica atrás de CNNs do mesmo porte, e que a vantagem aparece com mais dados.
- Os pesos usados aqui (`IMAGENET1K_V1`) foram treinados com classificação supervisionada na ImageNet-1k.

Diferenças que importam

- **Custo dos dados.** BERT aproveita texto bruto. O ViT supervisionado depende de rótulos, que são caros.
- **Onde está o sinal.** BERT recebe um alvo para cada token escondido, então aprende sobre cada parte da frase. O ViT recebe um alvo por imagem inteira.
- **O que a representação guarda.** O ViT supervisionado guarda o que separa as classes do pré-treinamento. Na ImageNet, isso é o objeto da foto. O estilo da imagem pode ser descartado, porque não ajuda a dizer se a foto é de um cão ou de um carro.
- **Ligação com este trabalho.** A tarefa aqui é quase o oposto: o objeto importa pouco e o estilo importa muito. Isso ajuda a entender por que o fine-tuning parou em 47%.

O equivalente de BERT em imagens: masked image modeling

- **BEiT:** esconde patches e prevê, para cada um, um token visual discreto produzido por um tokenizador de imagens.
- **MAE:** esconde a maior parte dos patches e reconstrói os pixels que faltam. O encoder só processa os patches visíveis.
- Os dois são self-supervised e dispensam rótulos, como BERT.
- O que maximizam: o acerto na previsão das partes escondidas (tokens visuais no BEiT, pixels no MAE). Para reconstruir um pedaço de pintura, textura e cor importam, o que parece mais próximo do que a datação precisa. Isso é uma hipótese e não foi testado aqui.

DeiT e Swin Transformer

DeiT: o problema dos dados

- Limite do ViT original: precisa de conjuntos enormes de imagens rotuladas para superar CNNs, porque não traz as suposições de uma CNN sobre imagens e precisa aprender tudo dos dados.
- O que o DeiT resolve: treinar um ViT competitivo só com a ImageNet-1k, sem dados extras.
- Como:
  - augmentation forte e regularização forte no treino;
  - um distillation token, que entra na sequência ao lado do CLS token. Ele aprende a imitar a resposta de um modelo professor, em geral uma CNN. O aluno herda parte das suposições da CNN.
- O encoder é o do ViT, com um token a mais. O que muda é a receita de treino.
- **Ligação com este trabalho.** O ViT do zero daqui está no regime que o DeiT ataca, em escala bem menor: 11.200 imagens, augmentation leve, nenhum professor, e o overfitting descrito em Resultados. A receita do DeiT indica o que tentar: augmentation mais forte e distillation a partir de um modelo melhor, como o ViT-B/16 com fine-tuning.

Swin Transformer: o problema da escala e do custo

- Limites do ViT original:
  - uma única grade de patches, com a mesma resolução do primeiro ao último bloco. Não há mapas de features em várias escalas;
  - attention global, de todos os tokens com todos. O custo cresce com o quadrado do número de tokens, o que pesa em imagens grandes.
- O que o Swin resolve:
  - **Hierarquia.** Começa com patches pequenos e junta patches vizinhos a cada estágio. A resolução cai e a dimensão sobe, como nos estágios de uma CNN.
  - **Janelas locais.** A attention é calculada dentro de janelas de tamanho fixo. O custo passa a crescer de forma linear com o tamanho da imagem.
  - **Janelas deslocadas.** De uma camada para a seguinte, as janelas mudam de lugar. Isso liga janelas vizinhas e deixa a informação atravessar a imagem.
- Resultado: um backbone de uso geral, que serve também para detecção e segmentação, tarefas que pedem features em várias escalas.
- **Ligação com este trabalho.**
  - O ViT daqui tem uma grade única de 8x8 patches, sem hierarquia. Cada patch de 8x8 pixels mistura pincelada, contorno e cor em um só vetor desde a primeira camada. É o que o Swin muda.
  - O custo quadrático não pesou aqui: são 65 tokens, ou 65 x 65 = 4.225 pares por head. Em uma pintura em alta resolução, onde a pincelada aparece, pesaria.

Quando usar ViT e quando usar CNN

O que cada arquitetura supõe

- **CNN:** supõe que pixels vizinhos se relacionam e que o mesmo filtro serve em qualquer lugar da imagem. Essas suposições valem para a maioria das imagens e economizam dados.
- **ViT:** supõe quase nada. Qualquer patch pode se relacionar com qualquer outro desde o primeiro bloco. Ganha flexibilidade e precisa de mais dados para aprender o que a CNN já traz pronto.

ViT é a melhor escolha quando

- existe muito dado, ou um pré-treinamento grande que possa ser aproveitado;

- a tarefa depende de relações entre partes distantes da imagem, como a composição da cena;
- o modelo vai ser combinado com texto ou outras modalidades, onde o Transformer é a arquitetura comum.

CNN é preferível quando

- o dataset é pequeno e não há bons pesos pré-treinados para o domínio;
- a pista é local: textura, borda, padrão que se repete;
- o limite é memória, tempo ou imagem em alta resolução.

Neste domínio

- Na Atividade 3, uma ResNet-50 pré-treinada, com o backbone congelado, chegou a 100% em um teste de 265 fotos de 7 classes de objetos, um domínio próximo da ImageNet. Aqui o ViT-B/16 pré-treinado ficou em 47,1%. A datação de pinturas é distante, e boa parte da dificuldade vem do domínio e da tarefa.
- As pistas de data são de dois tipos:
  - locais: pincelada, craquelado, textura da tinta. Favorecem a CNN;
  - globais: composição, paleta, uso do espaço. Favorecem o ViT.
- O experimento sem positional encoding sugere que, a 64x64, o ViT do zero não tirou proveito do arranjo espacial.
- **O que não foi medido.** Nenhuma CNN foi treinada nas pinturas, então este trabalho não compara o ViT-B/16 com uma ResNet-50 com fine-tuning.
- **Recomendação para este domínio:**
  - treinar do zero com cerca de 11 mil imagens: uma CNN pequena é a aposta mais segura, pelas suposições que economizam dados. É uma expectativa, sem medida aqui;
  - com pesos pré-treinados disponíveis: o ViT-B/16 com fine-tuning é a escolha apoiada pelos dados deste trabalho.

Análise crítica e próximos passos

Limites do experimento

- **Uma seed por modelo.** A variação entre execuções não foi medida. A diferença de 14,6 pontos entre o modelo pré-treinado e o do zero é grande o bastante para não depender disso. A comparação com e sem posição (1 pintura de diferença) depende.
- **Uma amostra.** São 16.000 pinturas sorteadas uma vez, de 76.881 com ano. Outra amostra daria outros pintores no teste.
- **Grupos pequenos.** Alguns grupos de movimento têm pouco mais de 20 pinturas no teste.
- **Fine-tuning curto e simples.** Duas épocas, sem augmentation, taxa de aprendizado constante. Um treino mais cuidadoso provavelmente passaria de 47,1%, mas isso não foi medido.
- **Rótulos.** Os anos não foram conferidos, e estilo e data podem divergir, como descrito em Problema.
- **Períodos de larguras diferentes.** Errar por 1 período depois de 1850 é errar cerca de 25 anos. Entre "até 1599" e "1600 a 1799" pode ser errar séculos. A distância em períodos não é uma distância em anos.

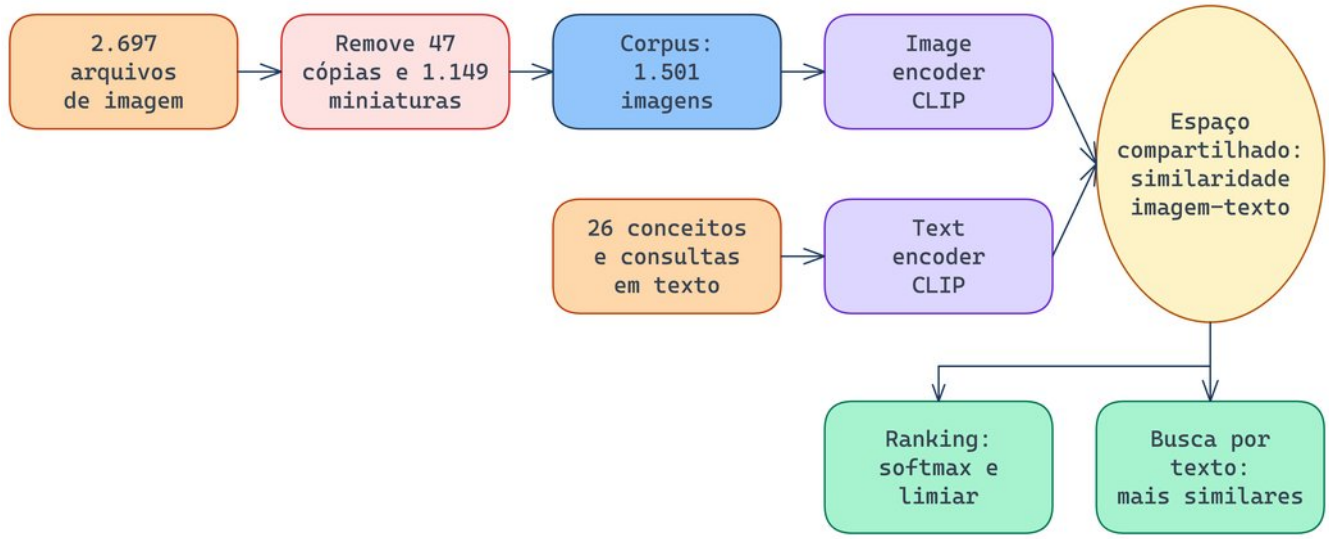
O que mudaria

- Repetir cada modelo com 3 a 5 seeds e informar média e desvio.
- Guardar os pesos da melhor época de validação, em vez dos da última.
- No fine-tuning: augmentation leve, queda da taxa de aprendizado e mais épocas com parada pela validação.
- Olhar o período 1875 a 1899 de perto: ver quais pintores e movimentos caem nele no treino e no teste.
- Treinar o ViT do zero a 128x128 ou mais, com augmentation mais forte, e repetir o teste sem posição. A hipótese a testar: com mais resolução e menos overfitting, a posição passa a fazer diferença.

- Fazer o fine-tuning de uma ResNet-50 nas mesmas pinturas, para ter a comparação com uma CNN neste domínio.
- Testar pesos de masked image modeling (MAE) como ponto de partida.
- Tratar a tarefa como ordinal ou como regressão do ano, já que os períodos têm ordem e errar por um vizinho custa menos que errar por séculos.
- Guardar o ano de cada pintura do teste, para medir o erro em anos e conferir a hipótese dos movimentos longos.

Atividade 2: Reconhecimento semântico em publicidade com CLIP

Problema



Atividade 2: do corpus à busca por texto

- Tarefa: usar o CLIP pré-treinado para ordenar conceitos pela frequência no corpus ADS-16 e para buscar imagens a partir de uma frase.
- Dataset: `groffo/ads16-dataset`, público no Kaggle. Modelo: `openai/clip-vit-base-patch32`, sem ajuste de pesos e sem uso de rótulos do ADS-16.

O corpus como veio

- O download tem 2.697 arquivos de imagem, todos legíveis, em duas partes: pastas `Ads`, com anúncios, e pastas `Corpus`, com imagens enviadas pelos participantes do estudo que gerou o dataset.
- Os anúncios estão em 20 pastas numeradas por categoria: 19 com 15 anúncios e 1 com 16.
- 47 cópias exatas foram removidas, comparando o conteúdo dos arquivos.
- 1.149 miniaturas foram removidas pela regra abaixo.

Parte	Imagens	Parcela
Anúncios	301	20,1%
Imagens de participantes	1.200	79,9%
Total	1.501	100%

Miniaturas: a regra e o seu limite

- Nas pastas dos participantes, um arquivo com `_th_` no nome é a cópia reduzida de outro arquivo da mesma pasta: `4_th_1448374800.png` é a miniatura de `4.png`.
- Regra: a miniatura sai quando o original está no corpus. Sem a regra, a mesma figura contaria duas vezes na frequência e viria em dobro nas buscas.
- Conferência: os 1.173 pares formados pelo nome foram comparados pelos pixels (conferido em uma análise à parte, fora do notebook). Em 1.159 as duas imagens são a mesma figura. Dos 1.173 pares, 24 miniaturas já tinham saído como cópias exatas. Por isso a regra removeu 1.149.

- Limite: a regra olha só o nome. Em 14 pares as figuras são diferentes, e essas 14 imagens ficaram fora do corpus.

Como o CLIP alinha imagem e texto

O que o pré-treinamento faz

- O CLIP tem dois encoders: um para imagens (aqui um ViT-B/32) e um para texto (um Transformer). Uma projeção no fim de cada um leva as duas saídas para o mesmo espaço de embedding.
- O treino usou cerca de 400 milhões de pares de imagem e legenda coletados na web. Ninguém anotou classes. A legenda que já acompanhava a imagem serve de supervisão.
- O objetivo é contrastivo. Em um batch com N pares, o modelo calcula a cosine similarity de cada imagem com cada texto. A perda aumenta a similaridade dos N pares corretos e reduz a dos pares trocados.
- Antes da softmax, as similaridades são multiplicadas por um fator aprendido (logit scale, o inverso da temperatura). Neste modelo o valor é 100.
- Resultado: uma imagem e um texto que combinam ficam próximos, e a cosine similarity entre os dois vetores mede esse acordo.

Por que isso permite busca sem treino supervisionado

- A legenda da web é texto livre. O modelo aprende a ligar imagens a frases, sem lista fechada de categorias, e qualquer frase nova já tem um lugar no espaço.
- Classificação zero-shot: cada classe vira um prompt, e a imagem recebe a classe do prompt mais próximo. Os prompts fazem o papel dos pesos de um classificador, sem nenhum exemplo rotulado.
- Busca semântica é a mesma conta no sentido contrário: uma frase, muitas imagens, e a resposta são as imagens de maior similaridade.

Neste projeto

- Cada uma das 1.501 imagens passou uma vez pelo encoder de imagem e virou um vetor de comprimento 1. Isso levou 37,5 segundos.
- Os mesmos embeddings servem ao ranking e à busca. Muda só o texto comparado: 26 prompts de conceito no ranking, 11 frases na busca.

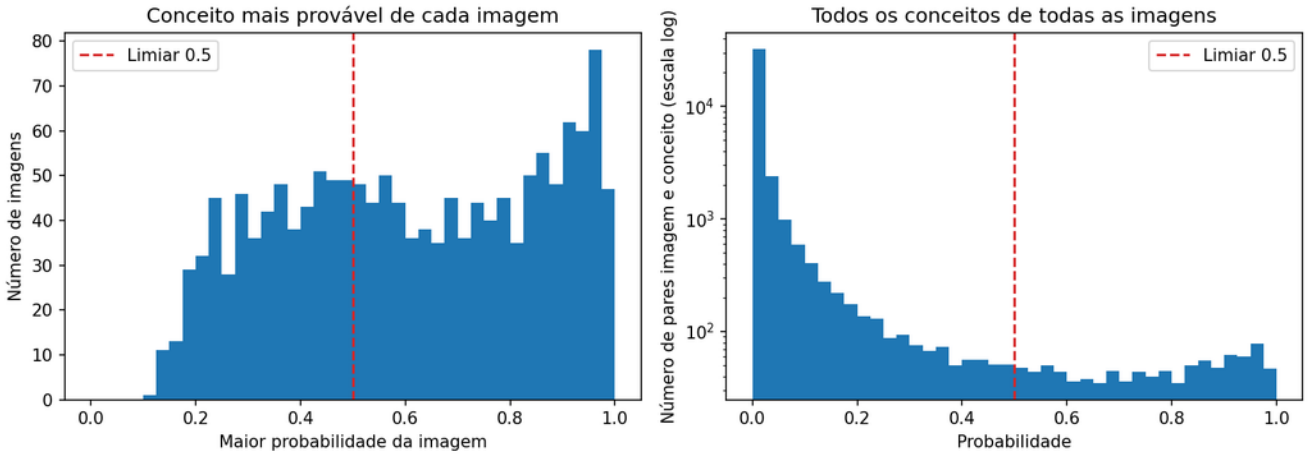
Ranking de conceitos

Prompts

- 26 conceitos, cada um escrito como "a photo of" seguido do conceito. A lista está na tabela do ranking.
- Ela cobre objetos de anúncio, pessoas, cenas e formatos de imagem. Foi fixada antes de ver os resultados, com uma redação por conceito, em inglês.

Regra de presença e escolha do limiar

- Para cada imagem, as 26 similaridades são multiplicadas por 100 e passam por uma softmax. Cada conceito recebe uma probabilidade e as 26 somam 1.
- Regra: um conceito está presente na imagem quando a sua probabilidade é igual ou maior que 0,5. A frequência é a parcela das imagens em que ele está presente.



Distribuição da maior probabilidade por imagem e de todas as probabilidades

- A distribuição não aponta um limiar. A figura não mostra vale: a maior probabilidade de cada imagem se espalha de 0,2 a 1, e os quartis são 0,404, 0,602 e 0,832.
- A escolha veio de olhar 8 imagens sorteadas logo abaixo e 8 logo acima de três candidatos: 0,3, 0,5 e 0,7. Nos 8 exemplos de cada lado vistos no notebook, perto de 0,3 os erros são comuns e perto de 0,7 a maioria dos rótulos combina.
- Em 0,5 a maioria combina. Acima da linha, os erros vistos são uma chave como "a car" e uma maquete de castelo como "a house". Logo abaixo há rótulos certos que a regra descarta, como um macaco em "an animal" com 0,46 (conferido em uma análise à parte, fora do notebook).



Imagens sorteadas abaixo (linha de cima) e acima (linha de baixo) do limiar de probabilidade 0,5

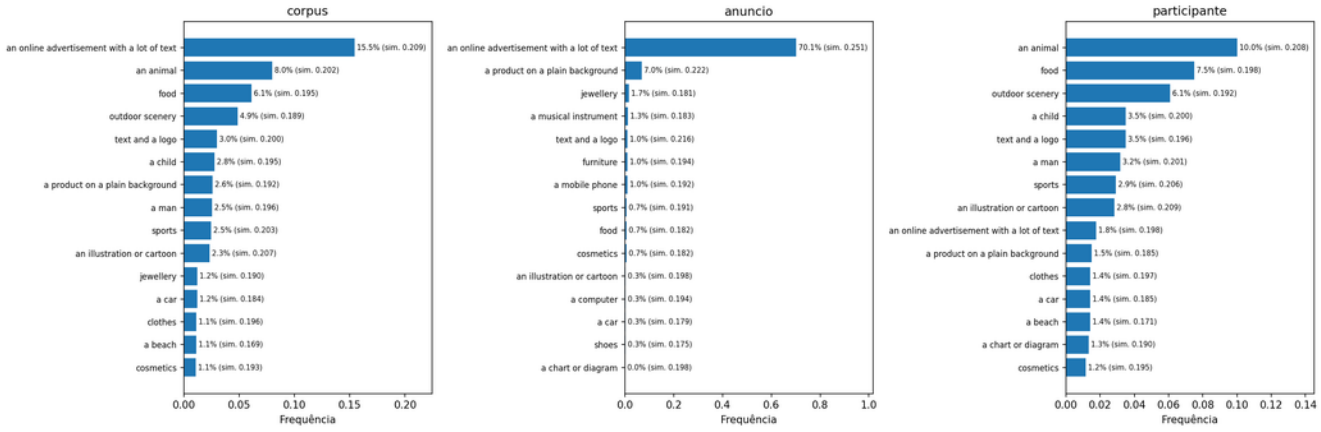
- Decisão: 0,5, a regra de maioria. O conceito tem mais probabilidade que os outros 25 juntos. É uma escolha de julgamento, sobre uma amostra pequena avaliada por mim.
- Custo: a parcela de imagens com algum conceito presente é 86,3% em 0,3, 62,6% em 0,5 e 40,0% em 0,7.

Resultado

Frequência com limiar 0,5. "Imagens" é a contagem no corpus inteiro. Empates são desfeitos pela similaridade média.

#	Conceito	Imagens	Corpus	Anúncios	Participantes
1	an online advertisement with a lot of text	232	15,5%	70,1%	1,8%
2	an animal	120	8,0%	0,0%	10,0%
3	food	92	6,1%	0,7%	7,5%

#	Conceito	Imagens	Corpus	Anúncios	Participantes
4	outdoor scenery	73	4,9%	0,0%	6,1%
5	text and a logo	45	3,0%	1,0%	3,5%
6	a child	42	2,8%	0,0%	3,5%
7	a product on a plain background	39	2,6%	7,0%	1,5%
8	a man	38	2,5%	0,0%	3,2%
9	sports	37	2,5%	0,7%	2,9%
10	an illustration or cartoon	35	2,3%	0,3%	2,8%
11	jewellery	18	1,2%	1,7%	1,1%
12	a car	18	1,2%	0,3%	1,4%
13	clothes	17	1,1%	0,0%	1,4%
14	a beach	17	1,1%	0,0%	1,4%
15	cosmetics	16	1,1%	0,7%	1,2%
16	a chart or diagram	16	1,1%	0,0%	1,3%
17	a person	12	0,8%	0,0%	1,0%
18	a mobile phone	12	0,8%	1,0%	0,8%
19	a smiling woman	10	0,7%	0,0%	0,8%
20	furniture	9	0,6%	1,0%	0,5%
21	a musical instrument	9	0,6%	1,3%	0,4%
22	a drink	9	0,6%	0,0%	0,8%
23	a house	9	0,6%	0,0%	0,8%
24	shoes	7	0,5%	0,3%	0,5%
25	a computer	4	0,3%	0,3%	0,2%
26	a city street	4	0,3%	0,0%	0,3%

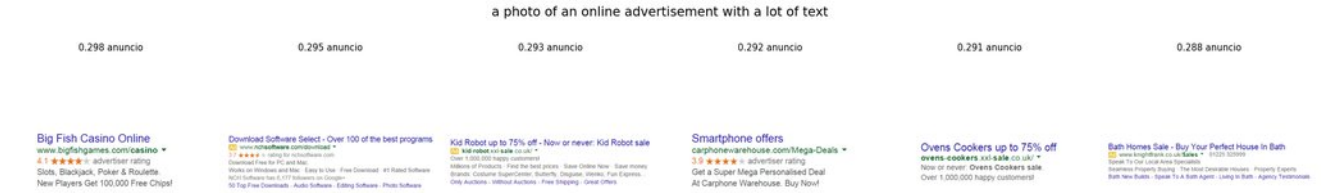


Os 15 conceitos mais frequentes no corpus, nos anúncios e nas imagens de participantes

- A similaridade média fica entre 0,162 e 0,209 nos 26 conceitos e quase não os separa.

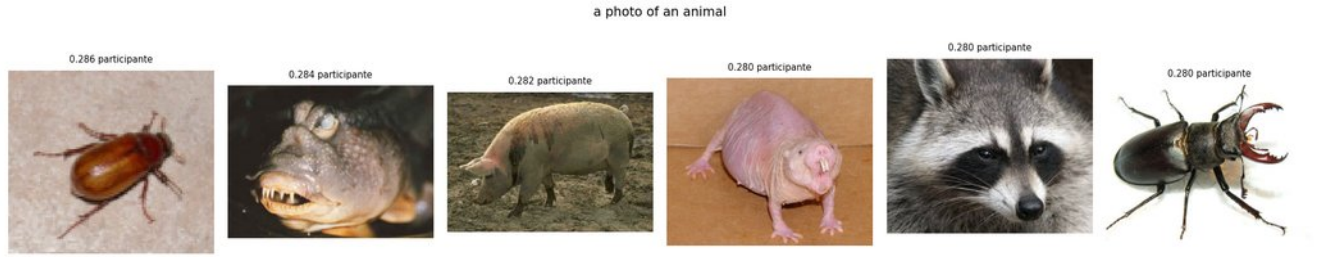
Os cinco conceitos mais frequentes

Cada figura mostra as seis imagens de maior similaridade com o prompt.



Imagens do conceito 1: an online advertisement with a lot of text

- Confirmado: seis anúncios de busca, só com título, endereço e linhas de texto.



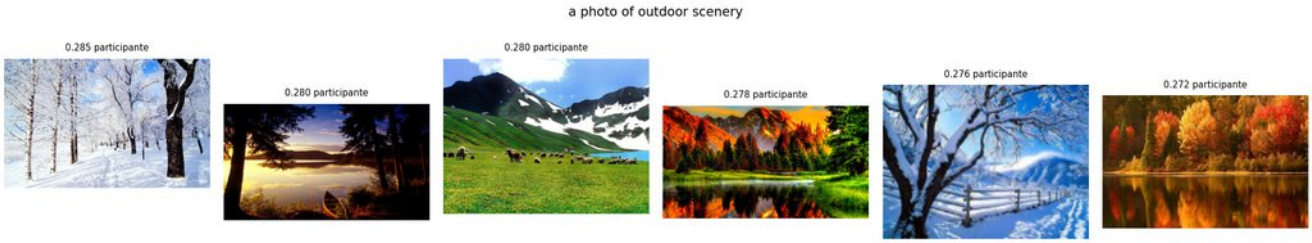
Imagens do conceito 2: an animal

- Confirmado: seis animais, entre eles dois besouros, um porco e um guaxinim.



Imagens do conceito 3: food

- Confirmado: seis pratos e alimentos, entre eles um peixe, um lanche com batata frita e uma pizza.



Imagens do conceito 4: outdoor scenery

- Confirmado: seis paisagens sem pessoas, com neve, lago, montanha e floresta.



Imagens do conceito 5: text and a logo

- Confirmado em parte. Três logotipos (imgur, THE 1975, eBay) e um anúncio de texto combinam.
- A imagem de maior similaridade é um retângulo todo preto. A sexta é uma arte abstrata colorida, sem texto.

O que o ranking diz e onde a regra falha

Sobre o corpus

- Os anúncios do ADS-16 são, em sua maioria, anúncios de texto. O prompt de anúncio com muito texto está presente em 211 dos 301. O segundo lugar entre os anúncios, "a product on a plain background", tem 21.
- Objetos anunciados quase não aparecem como conceito presente nos anúncios: "jewellery" tem 5 imagens, "a car" tem 1. Isso depende também da regra e da lista de prompts.
- O primeiro lugar do corpus inteiro depende da mistura das duas partes, por isso a tabela traz uma coluna para cada uma.

Onde a regra falha

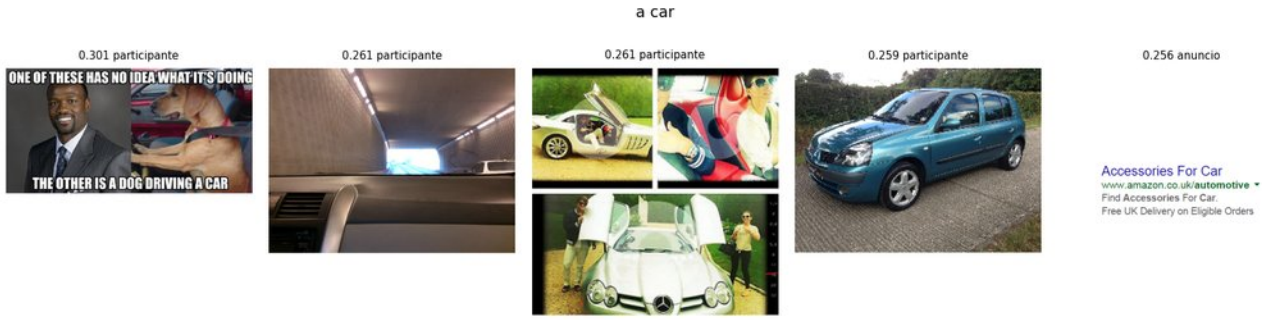
- A frequência mede quantas vezes um conceito vence a disputa entre os 26 prompts. Quantas imagens mostram cada objeto é outra pergunta, que exigiria rótulos.
- Uma imagem com um homem e um carro conta para um dos dois ou para nenhum.
- 561 imagens (37,4%) ficam sem conceito. Têm um conceito 260 dos 301 anúncios e 680 das 1.200 imagens de participantes.
- Prompts parecidos dividem a probabilidade. Os quatro prompts de pessoas ("a person", "a man", "a child", "a smiling woman") somam 15,2% de probabilidade média, mas só 102 imagens (6,8%) têm um deles presente. Os dois prompts de texto disputam da mesma forma. A regra conta pessoas e texto para menos.
- Efeito visível: "a person" fica em 17º lugar, abaixo de "a child" e de "a man".

Busca por texto

- 11 consultas, fixadas antes de ver os resultados. Nenhuma foi trocada depois.
- Cada consulta vira um embedding de texto. O resultado são as 5 imagens de maior cosine similarity entre as 1.501.

- A busca sempre devolve 5 imagens, mesmo quando nenhuma combina com a frase.

1. "a car"



Imagens recuperadas pela busca "a car"

- Há carro ou referência a carro nas cinco imagens. Ele é o assunto em duas: a colagem de um carro esportivo prateado e o carro azul.
- As outras são inesperadas: um meme com a frase "a dog driving a car", um túnel visto de dentro de um carro e um anúncio de texto com "Accessories For Car".

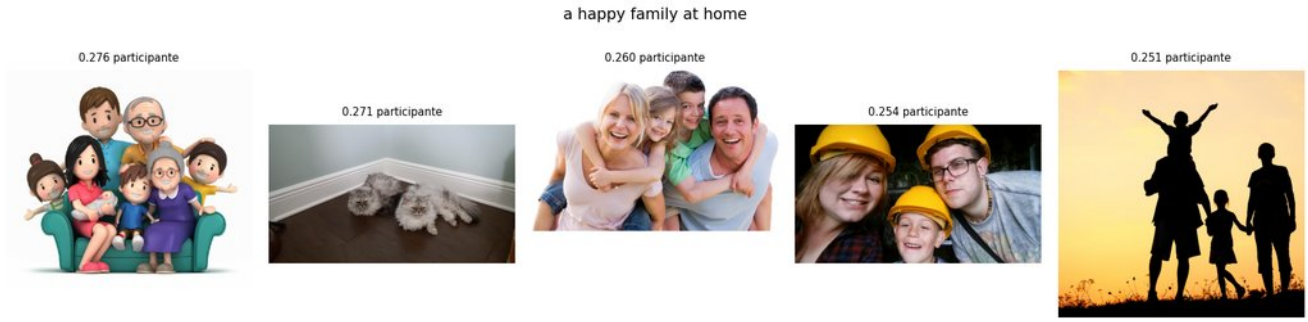
2. "a red sports car on a road"



Imagens recuperadas pela busca "a red sports car on a road"

- Nenhuma imagem mostra um carro esportivo vermelho em uma estrada.
- Cada resultado atende a um pedaço da frase: um conversível esportivo cinza, um carro amarelo em uma rua alagada, o meme do cachorro ao volante, crianças em um carrinho vermelho de brinquedo e um Mini vermelho sobre fundo preto.
- O primeiro resultado tem 0,245, o menor valor entre os primeiros resultados das 11 consultas.

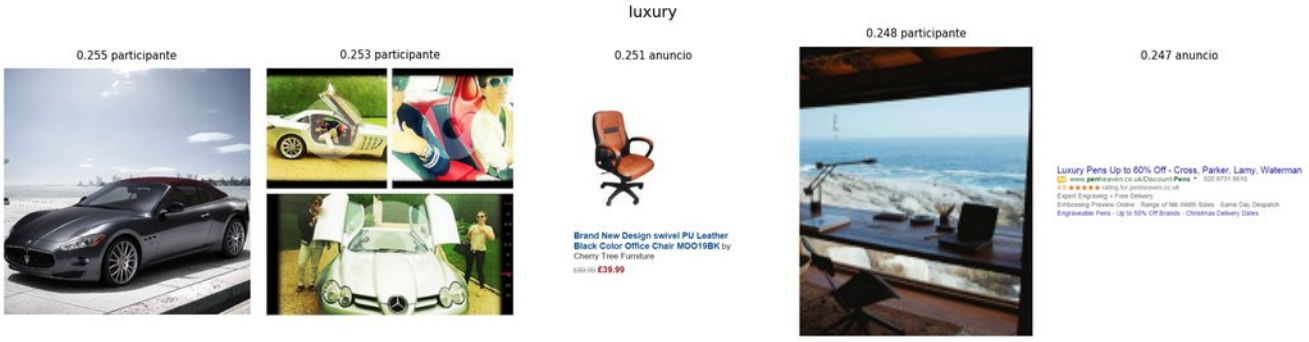
3. "a happy family at home"



Imagens recuperadas pela busca "a happy family at home"

- Quatro resultados são famílias alegres: um desenho 3D no sofá, uma foto sobre fundo branco, três pessoas de capacete amarelo e uma silhueta ao pôr do sol. Nenhuma foto de família é dentro de casa.
- O segundo resultado são dois gatos no canto de uma sala. Combina com "at home". Falta a família.

4. "luxury"



Imagens recuperadas pela busca "luxury"

- Os dois primeiros resultados são carros: o conversível cinza e a colagem do carro esportivo.
- O quinto é um anúncio de texto com "Luxury Pens" no título. A palavra escrita puxou a imagem.
- Os outros são um anúncio de cadeira de escritório de 39,99 libras e uma mesa diante de uma janela para o mar.

5. "freedom"



Imagens recuperadas pela busca "freedom"

- O primeiro resultado é literal: uma foto em preto e branco de uma multidão sobre um muro com a palavra FREEDOM pichada.
- Os outros são símbolos comuns de liberdade: cabelo ao vento, dois cavalos pretos correndo e uma pessoa de snowboard na neve.

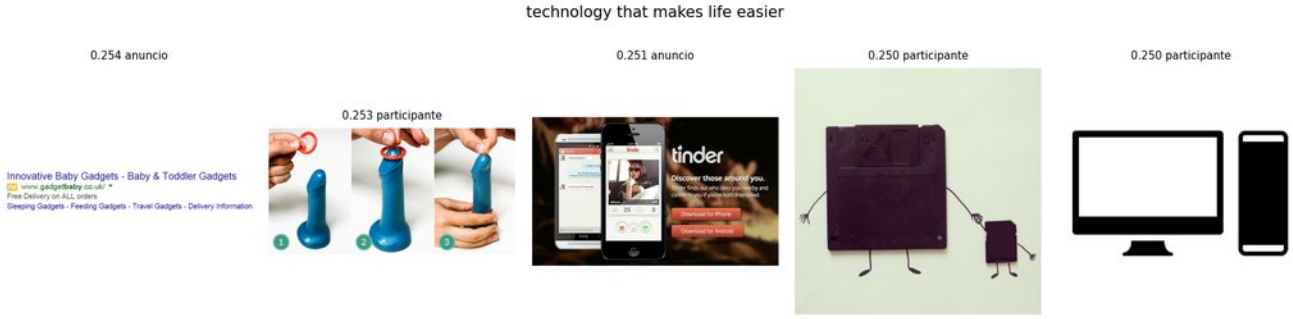
6. "something healthy to eat"



Imagens recuperadas pela busca "something healthy to eat"

- Os cinco resultados combinam: um prato de frutas, cenouras, uma pirâmide alimentar e brócolis.
- A descrição é indireta, sem citar fruta nem verdura, e o modelo a resolveu.
- O brócolis aparece duas vezes, uma em baixa resolução, em arquivos sem ligação pelo nome.

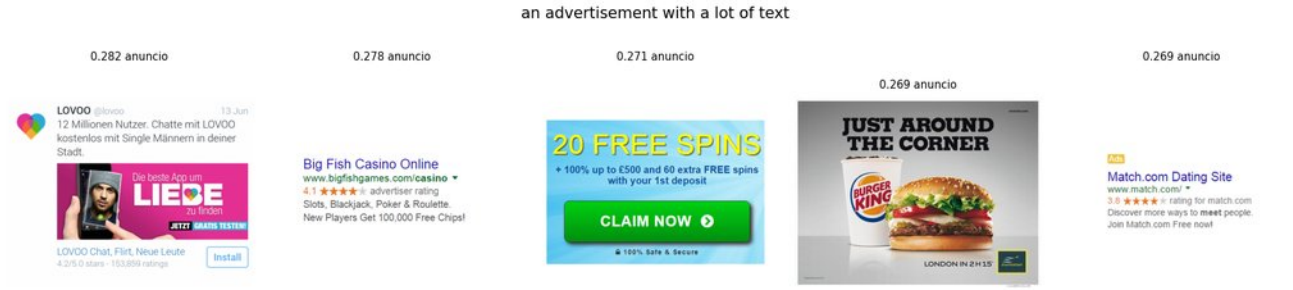
7. "technology that makes life easier"



Imagens recuperadas pela busca "technology that makes life easier"

- Resultado fraco. As cinco similaridades ficam entre 0,250 e 0,254, quase empatadas.
- Quatro têm ligação com tecnologia: um anúncio de texto de "Innovative Baby Gadgets", um anúncio do aplicativo Tinder, um disquete com braços desenhados e um ícone de computador.
- O segundo resultado é um passo a passo em três fotos, com um objeto azul de silicone e um anel vermelho, sem relação visível com a frase.

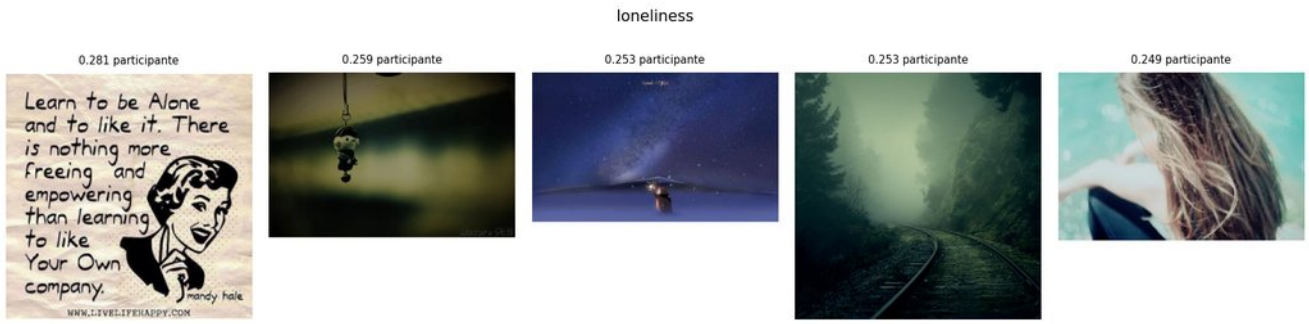
8. "an advertisement with a lot of text"



Imagens recuperadas pela busca "an advertisement with a lot of text"

- Os cinco resultados são anúncios, todos da parte de anúncios do corpus.
- Quatro têm bastante texto. O banner de hambúrguer tem só uma frase curta.

9. "loneliness"



Imagens recuperadas pela busca "loneliness"

- O primeiro resultado é de novo texto: um cartaz com a frase "Learn to be Alone and to like it".

- Seguem cenas vazias e escuras: um boneco pendurado, um céu estrelado com uma figura pequena e um trilho de trem na névoa.
- O quinto é a mesma foto de cabelo ao vento de "freedom". Uma imagem que serve a duas ideias indica uma ligação vaga.

10. "um cachorro brincando na praia"



Imagens recuperadas pela busca "um cachorro brincando na praia"

- Quatro resultados mostram cachorros. O modelo entendeu "cachorro" em português.
- Só o terceiro tem praia: um homem, uma criança e cachorros na areia. Outros três contrariam "brincando": um cachorro magro preso a uma corrente, um com o focinho amarrado com fita e um de coleira com pontas.
- O quinto é um borrão escuro e pixelado. Não dá para dizer que é um cachorro.

11. "uma pessoa sorrindo"



Imagens recuperadas pela busca "uma pessoa sorrindo"

- Três resultados são pessoas sorrindo: uma mulher em uma varanda, um homem com restos de folha verde nos dentes e um rapaz loiro.
- No segundo, a mulher cobre a boca com um cobertor. O quarto é um rosto branco de meme de terror, com um sorriso largo: o modelo achou o sorriso e ignorou o resto.

Resumo por tipo de consulta

Concreta contra abstrata

- Consultas sobre coisas visíveis funcionam: "something healthy to eat" e "an advertisement with a lot of text". "a car" e "uma pessoa sorrindo" funcionam em parte.
- As abstratas ("luxury", "freedom", "loneliness", "technology that makes life easier") dão resultados fracos, literais ou simbólicos.
- A similaridade não separa os dois casos. O primeiro resultado de "loneliness" tem 0,281 e o de "a red sports car on a road" tem 0,245. Todas as similaridades das buscas ficam entre 0,235 e 0,301.

Genérica contra específica

- A genérica "a car" acha carros. Nas específicas ("a red sports car on a road", "a happy family at home", "um cachorro brincando na praia"), o assunto principal aparece e a cor, o lugar ou a ação ficam de fora na maior parte dos resultados.
- Parte disso é o corpus: com 1.501 imagens, a cena pedida pode não existir.

Inglês contra português

- As duas consultas em português acharam o assunto certo: cachorros e sorrisos.
- A perda de detalhes é parecida com a das consultas específicas em inglês. Com duas consultas, e sem a mesma frase nas duas línguas, este experimento não mede a diferença entre elas.

Consulta de texto no CLIP e tokenização no BERT

O que os dois tokenizers fazem com o mesmo texto

Três frases passaram pelo tokenizer do CLIP e pelo do `bert-base-uncased`, com padding até a mais longa. Saída para "a car":

- CLIP, tokens: `<|startoftext|>`, `a</w>`, `car</w>`, `<|endoftext|>` e mais 23 vezes `<|endoftext|>`.
- BERT, tokens: `[CLS]`, `a`, `car`, `[SEP]` e mais 23 vezes `[PAD]`.
- Attention mask, igual nos dois: 1, 1, 1, 1 e depois 23 zeros.

Frase	Tokens reais	Padding
"a car"	4	23
"a red sports car on a road"	9	18
Frase longa sobre uma família no café da manhã	27	0

- Os números da tabela são os mesmos nos dois tokenizers. Nestas frases, cada palavra virou um único token, e a vírgula da terceira frase também.
- Os dois passam o texto para minúsculas, cortam palavras raras em unidades menores e trocam cada unidade por um id do vocabulário. O CLIP usa byte-pair encoding e marca o fim de cada palavra com `</w>`. O BERT usa WordPiece.
- Os dois acrescentam dois tokens especiais: início e fim de texto no CLIP, `[CLS]` e `[SEP]` no BERT. O CLIP aceita até 77 tokens por texto e o BERT até 512.

Padding

- O modelo processa um batch como uma matriz, e todas as linhas precisam do mesmo comprimento. O padding completa as frases curtas até o tamanho da mais longa.
- O BERT tem um token próprio para isso, `[PAD]`. O tokenizer do CLIP repete o token de fim de texto.

Attention mask

- Ela marca os tokens reais com 1 e o padding com 0. A attention usa a máscara para tirar o peso das posições com 0.
- No BERT a attention é bidirecional: cada token olha para todos os outros. Sem a máscara, os 23 tokens de padding entrariam na conta, e a representação de "a car" mudaria conforme a frase mais longa do batch.
- O encoder de texto do CLIP usa também uma máscara causal: cada posição só olha para si e para as anteriores. Como o padding vem depois do fim do texto, ele já fica fora do alcance das posições reais.

Da sequência a um vetor

- CLIP: o vetor do texto é a saída na posição do token de fim de texto (a primeira ocorrência, já que o padding repete o mesmo token). Com a máscara causal, essa é a posição que viu a frase inteira. O vetor passa pela projeção e cai no espaço comum com as imagens, pronto para a cosine similarity.
- BERT: o vetor da sequência é a saída na posição do `[CLS]`, no início. Com attention bidirecional, a primeira posição também vê a frase inteira. Esse vetor alimenta uma classification head, treinada com rótulos na tarefa final.

Análise crítica

O modelo lê o que está escrito na imagem

- Em quatro consultas, algum resultado tem escrita a palavra pedida, ou uma próxima: "car" no meme e no anúncio, "Luxury" no anúncio de canetas, FREEDOM no muro, "Alone" no cartaz.
- Os anúncios do corpus são quase todos texto. Neles, a busca tende a achar o anúncio que cita a palavra antes do anúncio que mostra o objeto.

Viés do modelo e dos dados de treino

- Os resultados sugerem que as associações do CLIP podem refletir as das legendas da web. Aqui, "luxury" trouxe dois carros esportivos e "freedom" dois cavalos correndo.
- As legendas de treino são provavelmente em sua maior parte em inglês. As consultas em português usaram palavras comuns. Termos raros ou regionais e o viés sobre pessoas ficaram sem teste.

Prompts

- Há uma redação por conceito, em inglês. A ordem do ranking pode mudar com outra redação, e isso não foi medido.
- O molde "a photo of" foi aplicado a todos os conceitos, inclusive a banners e ilustrações.

Imagens que não são anúncios

- Para falar de publicidade, vale a coluna de anúncios. Nela a lista de 26 conceitos serve pouco, porque um único prompt de formato domina.

Sem gabarito

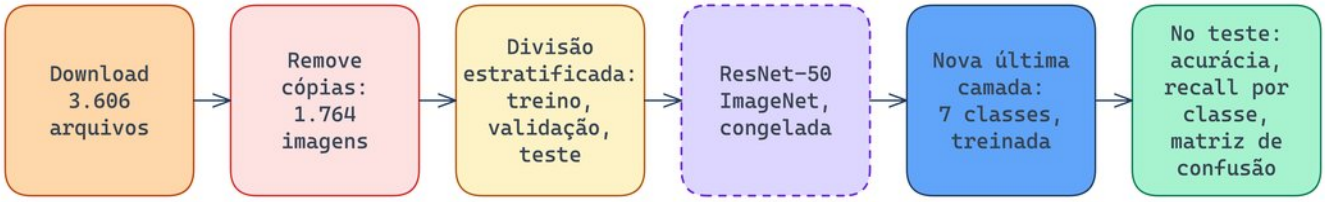
- O ADS-16 não traz rótulos de objeto, então não há precision nem recall.

O que eu mudaria

- Várias redações por conceito, com a média dos embeddings de texto.
- Decisão independente por conceito, comparando cada prompt com um prompt neutro, para que uma imagem possa ter mais de um conceito.
- Rotular à mão uma amostra de cerca de 200 imagens e medir a precision da regra em cada limiar.
- Remover miniaturas e repetições pela comparação de pixels, o que poderia pegar também o brócolis repetido.
- Um CLIP multilíngue para consultas em português.

Atividade 3: Classificador com CNN pré-treinada

Problema



Atividade 3: das fotos ao classificador

- Tarefa: dizer a qual de 7 classes pertence uma foto.
- Dataset: `pavansanagapati/images-dataset`, público no Kaggle.
- Classes: bike, cars, cats, dogs, flowers, horses, human. A classe é o nome da pasta da imagem.

O download traz o dataset repetido (conferido em uma análise à parte, fora do notebook)

- O arquivo do Kaggle tem 3.606 imagens em duas árvores de pastas com o mesmo conteúdo: `data/<classe>/` e `data/data/<classe>/`, com 1.803 arquivos cada.
- Comparando o conteúdo dos arquivos (SHA-256), existem 1.764 imagens distintas.
- 1.725 imagens aparecem 2 vezes, uma em cada árvore.
- 39 imagens aparecem 4 vezes, porque a mesma foto está guardada duas vezes dentro de cada árvore: 1 em bike, 19 em horses e 19 em human.
- Nenhuma imagem aparece em duas classes diferentes.
- O notebook remove as cópias exatas antes de dividir os dados: 1.842 arquivos removidos, 1.764 imagens usadas.
- Formatos dos 3.606 arquivos: 1.616 JPG, 1.570 BMP e 420 PNG.

Imagens distintas por classe

Classe	Imagens
bike	364
cars	420
cats	202
dogs	202
flowers	210
horses	183
human	183
Total	1.764

Divisão estratificada, seed 42

Conjunto	Imagens	Parte
Treino	1.234	70%
Validação	265	15%
Teste	265	15%

Decisões técnicas

- Modelo carregado: ResNet-50 do torchvision com os pesos `IMAGENET1K_V2`, pré-treinados na ImageNet.
- Classification head trocada: a camada final `fc`, que tinha 1.000 saídas, deu lugar a uma camada linear nova com 7 saídas, uma por classe do dataset.
- Feature extraction: todos os pesos do backbone ficam congelados (`requires_grad = False`). Só a camada nova recebe gradiente.
- Pré-processamento igual ao publicado para esses pesos: redimensionar o lado menor para 232, recorte central de 224x224, média e desvio padrão da ImageNet.
- Sem augmentation nesta execução. As propostas estão na seção "Melhorias propostas".

Por que ResNet-50

- Cabe com folga na T4. O pico de memória alocada pelo PyTorch na GPU foi de 465,8 MB, cerca de 3% dos 15,6 GB da placa.
- É rápida o bastante para o Colab. O notebook inteiro rodou em 115,3 segundos, contando download, 10 épocas e avaliação.
- A saída do backbone tem 2.048 features por imagem. Para separar 7 classes, a camada nova precisa de 2.048 x 7 pesos mais 7 vieses: 14.343 parâmetros.
- A ImageNet já contém fotos muito parecidas com as deste dataset: várias raças de cães e de gatos, tipos de carro, bicicletas, cavalos e flores. Pessoas aparecem em muitas fotos da ImageNet, embora ela não tenha uma classe "pessoa".
- Com 7 classes de objetos do dia a dia, um modelo maior não teria erro de teste para corrigir. A validação errou 2 imagens em 265.
- Comparação com a EfficientNet-B0, segundo a documentação do torchvision:
  - A EfficientNet-B0 é menor: cerca de 5,3 milhões de parâmetros e 1.280 features, com 77,7% de acurácia top-1 na ImageNet.
  - A ResNet-50 com `IMAGENET1K_V2` tem 80,9% de top-1. Como a memória e o tempo ficaram longe do limite da T4, a economia da EfficientNet-B0 não faria diferença aqui. Ela seria a escolha se o limite fosse memória ou tempo.

Por que feature extraction

- O dataset é pequeno: 1.234 imagens de treino.
- O domínio é próximo do da ImageNet: fotos comuns de animais, veículos, flores e pessoas.
- A rede tem 23.522.375 parâmetros. Treinar todos com 1.234 imagens daria cerca de 19 mil parâmetros por imagem, com risco alto de overfitting.
- Com o backbone congelado, só 14.343 parâmetros treinam: 0,06% do total.
- O treino fica barato. O backbone só faz a passagem direta e não guarda gradientes, o que explica o pico de memória baixo.

Hiperparâmetros

- **Otimizador: Adam.** Ajusta o passo de cada peso sozinho. Para uma única camada linear, funciona bem sem ajuste fino.
- **Taxa de aprendizado: 0,001.** É o valor padrão do Adam. A perda caiu em todas as épocas, sem oscilar, então não houve motivo para mudar.
- **Épocas: 10.** A acurácia de validação parou de mudar na época 2. As 10 épocas servem para mostrar a curva estável e custam pouco tempo.
- **Batch size: 32.** Dá 39 passos por época e usa pouca memória.
- **Backbone em modo de avaliação.** Em modo de treino, as camadas de batch normalization atualizam suas estatísticas mesmo com os pesos congelados. Em modo de avaliação, o backbone produz as mesmas features no treino, na validação e no teste.
- **Função de perda: entropia cruzada**, sem pesos por classe. A maior classe (cars, 420) tem 2,3 vezes o tamanho da menor (horses e human, 183), um desbalanceamento moderado.
- **Seed: 42**, usada na divisão dos dados e na inicialização da camada nova.

Resultados

Acurácia global no teste: 100% (265 acertos em 265 imagens).

Acurácia por classe no teste

Classe	Imagens no teste	Acertos	Acurácia	Queda com 1 erro
bike	54	54	100%	1,85 ponto
cars	63	63	100%	1,59 ponto
cats	30	30	100%	3,33 pontos
dogs	31	31	100%	3,23 pontos
flowers	32	32	100%	3,13 pontos
horses	28	28	100%	3,57 pontos
human	27	27	100%	3,70 pontos
Global	265	265	100%	0,38 ponto

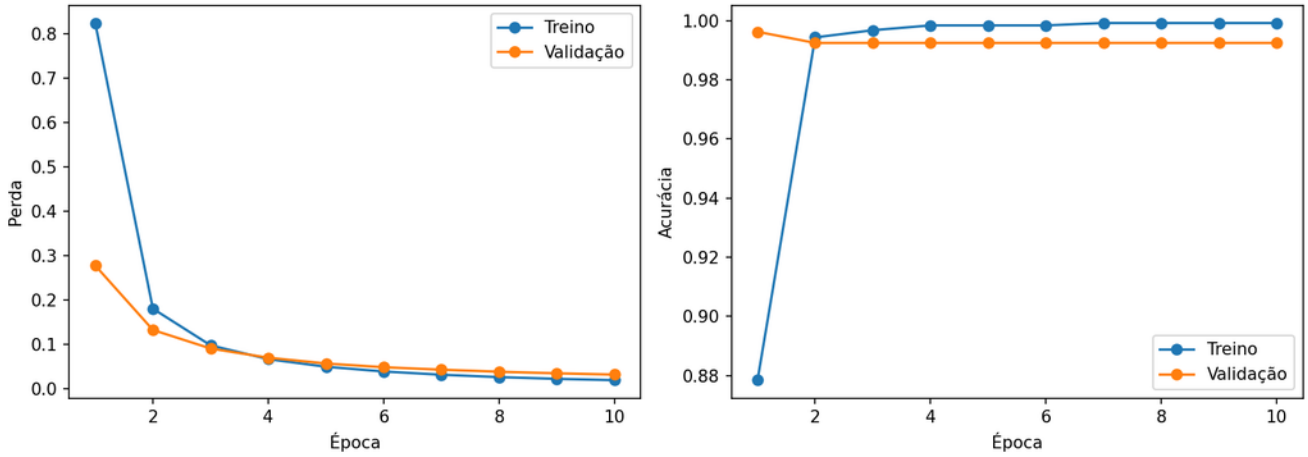
- A última coluna mostra quantos pontos percentuais a acurácia perderia com um único erro.

Curvas de treino

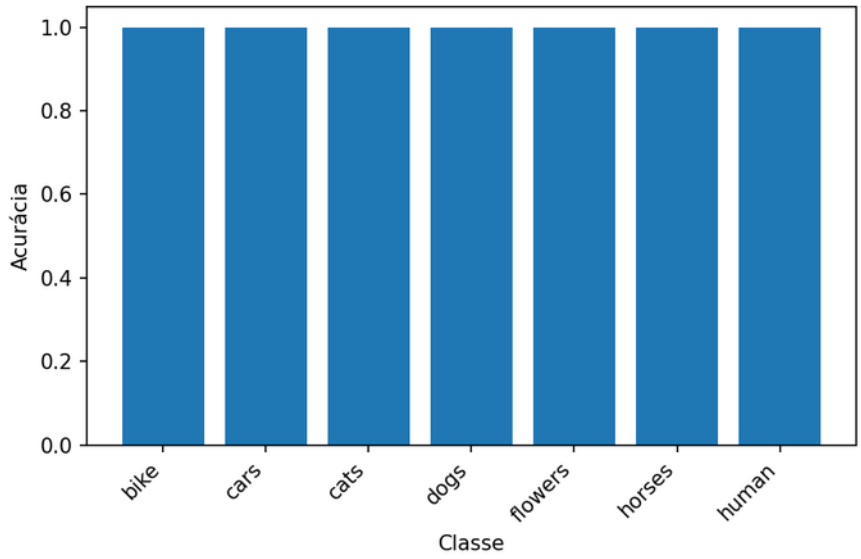
Época	Perda treino	Perda validação	Acurácia treino	Acurácia validação
1	0,824	0,277	87,84%	99,62%
2	0,180	0,132	99,43%	99,25%
5	0,049	0,056	99,84%	99,25%
10	0,018	0,031	99,92%	99,25%

- A acurácia de validação estabiliza na época 2, em 99,25% (263 de 265), e fica igual até a época 10.
- Na época 1 a validação acertou 264 de 265 (99,62%). Da época 2 em diante, 263.
- A perda de validação cai em todas as épocas, de 0,277 para 0,031. Não há subida que indique overfitting.

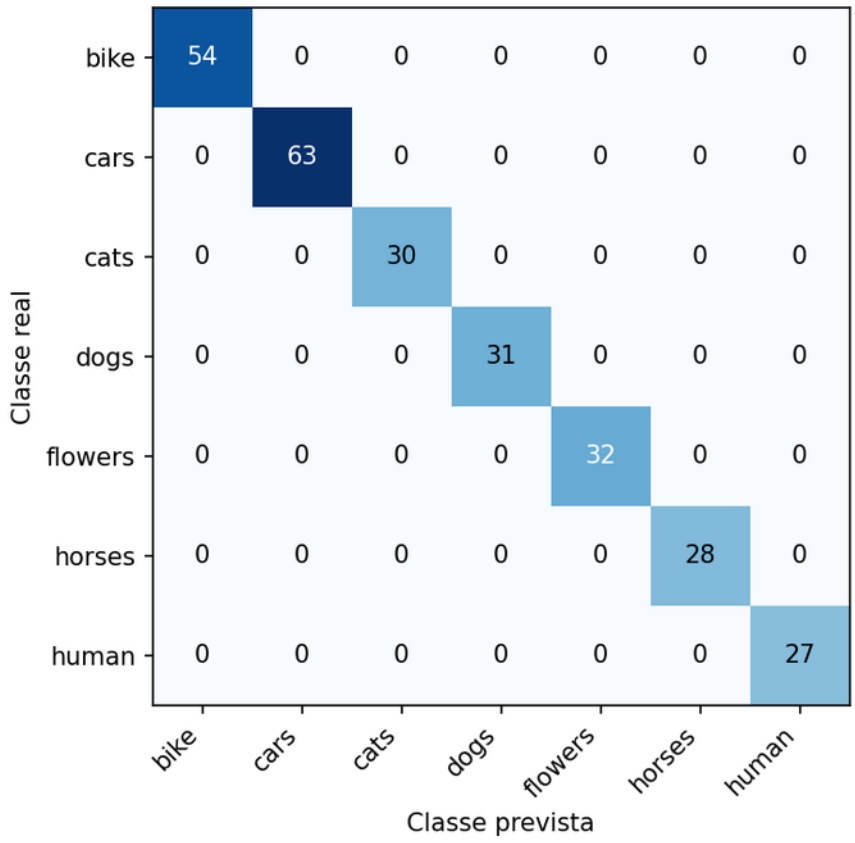
- Diferença final entre treino e validação: 0,67 ponto percentual na acurácia (99,92% contra 99,25%) e 0,013 na perda.
- Na época 1 o treino aparece pior que a validação (87,84% contra 99,62%). O número de treino é a média durante a época, enquanto a camada ainda está aprendendo. A validação é medida no fim da época.
- No treino, a última época errou 1 imagem em 1.234.



Curvas de perda e acurácia por época



Acurácia por classe no teste



Matriz de confusão no teste

Análise crítica

O que a matriz de confusão mostra

- A matriz de confusão do teste não tem nenhum erro. Não existe par de classes confundido para analisar.
- Os únicos erros medidos estão na validação: 2 imagens em 265.
- O notebook não registra quais são essas 2 imagens nem a classe delas. Sem isso, não dá para apontar a causa. Guardar as previsões da validação é uma melhoria simples para uma próxima execução.

Por que a tarefa é fácil para as features da ImageNet

- As 7 classes são bem diferentes entre si e quase todas têm classes parecidas na ImageNet.
- O backbone já separa esses objetos antes de qualquer treino. A camada nova só precisa aprender a qual das 7 saídas cada grupo de features pertence. Isso explica a validação acima de 99% já na primeira época.
- As classes também diferem no tipo de foto:
  - bike e cars são fotos de rua, quase todas com 640x480 pixels, em BMP.
  - flowers são closes de 128x128 pixels, em PNG, com a flor ocupando o quadro inteiro.
  - cats e dogs são fotos caseiras e de abrigo, em JPG.
  - horses e human são imagens pequenas da internet, em JPG. Em horses há desenhos e imagens geradas por computador. Em human, as pessoas usam roupa de equitação e muitas aparecem ao lado de um cavalo.
- Risco que isso traz: a classe anda junto com o formato e a resolução do arquivo. Uma flor de 128 pixels ampliada para 232 fica borrada. Uma foto de rua reduzida fica nítida. O modelo pode usar essa diferença como atalho. Esse efeito não foi medido.

O primeiro resultado foi descartado

- A primeira execução usou as 3.606 imagens, sem remover cópias, e chegou a 100% no teste.
- Nela, cópias da mesma foto caíam no treino e no teste. O modelo era avaliado em fotos que já tinha visto.
- Esse resultado foi descartado como inválido. Os números deste relatório vêm da execução com 1.764 imagens distintas.
- Limite da correção: só cópias exatas foram removidas. Quase-duplicatas, como a mesma foto redimensionada ou regravada com outra compressão, não foram procuradas. Se existirem, ainda pode haver vazamento entre treino e teste.

O que o resultado não mostra

- Foi uma única execução, com uma única seed e uma única divisão dos dados. A variação entre execuções não foi medida.
- O teste tem 265 imagens. Um erro tira 0,38 ponto da acurácia global.
- As classes do teste têm entre 27 e 63 imagens. Um erro em human (27 imagens) baixaria a classe para 96,30%. Um erro em cars (63 imagens) baixaria a classe para 98,41%.
- Com 0 erros em 265 imagens, o limite superior para a taxa de erro real é de cerca de 3/265 = 1,1%, com 95% de confiança (aproximação conhecida como rule of three). O resultado é compatível com uma acurácia real perto de 99%, como a da validação (99,25%).
- 100% em um teste pequeno indica que o modelo erra pouco neste dataset. Erros continuam possíveis, como os 2 da validação.
- O teste vem da mesma coleção do treino. O resultado não diz como o modelo se sairia em fotos de outra origem, por exemplo carros fotografados de perto ou pessoas com outro tipo de roupa.
- As imagens de human quase sempre mostram roupa de equitação e cenário de hípica. O modelo pode ter aprendido o cenário. O teste não separa isso da figura humana em geral.

Melhorias propostas

- Nenhuma das estratégias abaixo foi executada. A acurácia de teste já está no teto, então o ganho esperado é em fotos de outra origem. Neste teste não há erro para corrigir.
- Todas valem só para o conjunto de treino. Validação e teste continuam com o pré-processamento fixo.

Geometric augmentation

- **Flip horizontal: seguro para as 7 classes.** Nenhuma classe depende de esquerda ou direita.
  - Ajuda mais em bike, cars e horses, que aparecem muito de perfil, virados para um lado ou para o outro.
  - Em human, algumas fotos têm marca d'água com texto. O texto fica espelhado, mas ele não define a classe.
- **Rotação pequena, até 15 graus: ajuda cats e dogs.** Os animais aparecem deitados, no colo e em ângulos variados, e a câmera é de mão.
- **Rotação livre e flip vertical: só para flowers.** As flores são fotografadas de cima e de perto, sem um "lado de cima" definido.
- **Flip vertical e rotação grande distorcem bike, cars, horses e human.** Nessas fotos o chão está sempre embaixo, e as pessoas de human estão de pé. Uma rua de cabeça para baixo não existe no uso real.

Color augmentation

- **Brilho e contraste: ajudam bike e cars.** São fotos de rua com sol, sombra e algumas cenas noturnas com flash. A variação de luz já existe nos dados, e a augmentation cobre os casos intermediários.

- **Mudança de matiz (hue): livre para bike, cars e flowers.** Bicicletas e carros têm pintura de qualquer cor. As flores do dataset são vermelhas, cor-de-rosa, roxas, amarelas, brancas e laranja.
- **Mudança de matiz distorce cats, dogs e horses.** A pelagem fica entre branco, preto, marrom, cinza e laranja. Um cavalo verde ou um gato azul não existe. Para essas classes, a mudança de matiz deve ser pequena.
- **Tons de cinza com probabilidade baixa: ajuda horses e human.** Só essas duas classes têm imagens em preto e branco, e são poucas. Em human há também pelo menos uma foto antiga em sépia. Sem augmentation, o modelo pode aprender que "foto sem cor" quer dizer uma dessas duas classes. Aplicar tons de cinza às vezes em todas as classes quebra esse atalho.
- **Tons de cinza prejudica flowers.** A cor forte da pétala é a pista mais clara da classe. Sem cor, sobra só a forma. Por isso a probabilidade deve ser baixa, perto de 10%.

Scale variation

- Estado atual: o recorte central corta as bordas.
  - Em uma foto de 640x480 (bike e cars), o recorte de 224x224 mantém 72% da largura.
  - Em um retrato de human de 183x275, mantém 64% da altura. Cabeça e botas podem ficar de fora.
- **Recorte aleatório com mudança de escala (RandomResizedCrop): seguro para flowers.** A flor ocupa o quadro inteiro. Qualquer recorte ainda mostra pétalas.
- **Ajuda cats e dogs com recortes moderados.** O animal ocupa boa parte da foto, e a distância da câmera varia bastante.
- **Recorte agressivo prejudica cars e bike.** Em muitas fotos de cars, o carro é pequeno e está longe, em uma rua larga. Um recorte pequeno pode pegar só asfalto, prédio ou árvore, com o rótulo "cars". Como bike e cars têm o mesmo tipo de rua ao fundo, um recorte sem o objeto fica ambíguo entre as duas classes.
- **Recorte agressivo prejudica human.** A pessoa está de corpo inteiro e muitas vezes ao lado de um cavalo. Um recorte pequeno pode mostrar só o cavalo, com o rótulo "human", e empurrar o modelo para confundir human com horses.
- Proposta: recorte com área entre 60% e 100% da imagem, em vez do padrão de 8% a 100%.

Normalização

- **Já usada: média e desvio padrão da ImageNet, por canal.** Vale para as 7 classes e precisa continuar assim. O backbone está congelado, e seus filtros e camadas de batch normalization esperam entradas nessa escala.
- **Média e desvio padrão do próprio dataset: prejudica em feature extraction.** As features sairiam diferentes das do pré-treinamento, e o backbone não pode se ajustar. Só faria sentido com fine-tuning da rede inteira.
- **Normalizar também a resolução: afeta flowers, bike e cars.** flowers tem 128x128 pixels e é ampliada. bike e cars têm 640x480 e são reduzidas. Reduzir todas as imagens para um mesmo tamanho pequeno antes do pré-processamento tiraria a nitidez como pista da classe. É um teste direto para o atalho descrito na análise crítica.
- **Normalização por imagem (cada foto com sua própria média): distorce bike.** As fotos noturnas ficariam com o mesmo brilho médio das fotos de dia, e o ruído do escuro seria ampliado.

Feature extraction ou fine-tuning

Regra de decisão: tamanho do dataset contra distância do domínio em relação à ImageNet

	Domínio próximo da ImageNet	Domínio distante da ImageNet
Dataset pequeno	Feature extraction. Backbone congelado, treina só a classification head.	Caso mais difícil. Fine-tuning só dos últimos blocos, com taxa de aprendizado baixa e augmentation forte.
Dataset grande	Fine-tuning da rede inteira, com taxa de aprendizado baixa. As features já são boas e os dados permitem refinar.	Fine-tuning da rede inteira. Os pesos pré-treinados servem como ponto de partida.

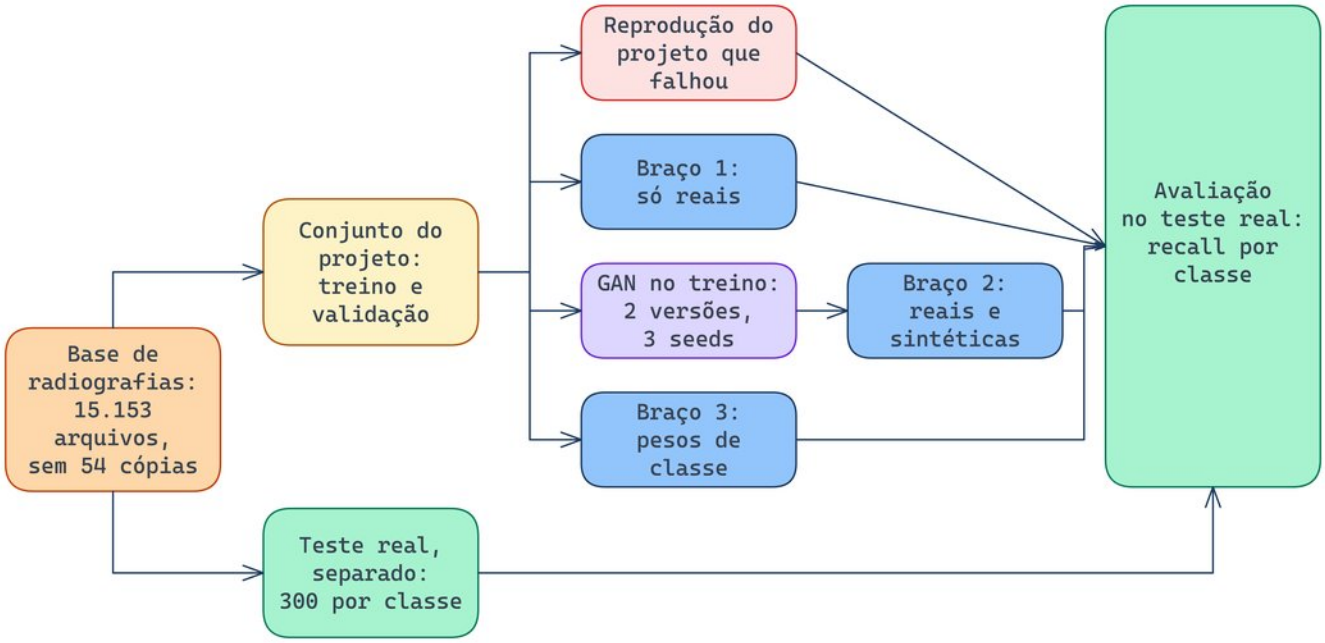
- **Feature extraction** serve quando as features prontas já separam as classes e há poucos dados. Poucos parâmetros treinam, então o risco de overfitting é baixo e o treino é rápido.
- **Fine-tuning** serve quando as features prontas não bastam. Isso acontece quando o domínio é diferente (raio-X, imagem de satélite, microscopia) ou quando as classes são muito parecidas entre si. Ele exige mais dados, porque milhões de pesos passam a treinar.
- Com poucos dados e domínio distante, as duas opções têm problema: as features prontas servem mal, e treinar muitos pesos leva a overfitting. As primeiras camadas (bordas, texturas) ainda costumam servir, por isso a saída é liberar só os últimos blocos.

Onde este dataset cai

- Tamanho: pequeno. São 1.764 imagens, 1.234 no treino.
- Domínio: próximo. São fotos comuns de objetos que a ImageNet já cobre.
- Célula da tabela: dataset pequeno e domínio próximo. A escolha é feature extraction.
- O resultado é coerente com a escolha: com 14.343 parâmetros treinados, a validação chegou a 99,25% e o teste a 100%.
- Fine-tuning teria pouco a ganhar aqui. Sobram 2 erros na validação e nenhum no teste. Em troca, colocaria 23.522.375 parâmetros para treinar com 1.234 imagens.

Atividade 4.1: Triagem de COVID-19 em radiografias de tórax

O projeto analisado



Atividade 4.1: dos dados à avaliação dos três braços

- Um grupo anterior montou um sistema de triagem de pneumonia e COVID-19 em radiografias de tórax e encerrou o projeto com resultados "promissores". O time clínico relatou que o modelo "ignora casos positivos".
- Dados: 1.200 radiografias em três classes: Normal (840), Pneumonia (240) e COVID-19 (120). Divisão 80/20 sem estratificação.
- Modelo: ResNet-18 sem pré-treinamento, SGD, taxa de aprendizado fixa de 0,01, 15 épocas, sem augmentation.
- Avaliação: só acurácia global. Resultado reportado: 93% no treino e 61% na validação.

O que este trabalho fez com o caso

- Reproduziu o projeto com as mesmas configurações, em imagens públicas, mediu o que o time não reportou e testou uma GAN condicional como resposta à falta de imagens de COVID-19.
- A reprodução não repete os 93% e 61%. As imagens são outras e a entrada tem 64x64 pixels. Ela serve para mostrar o que a métrica do time escondia.

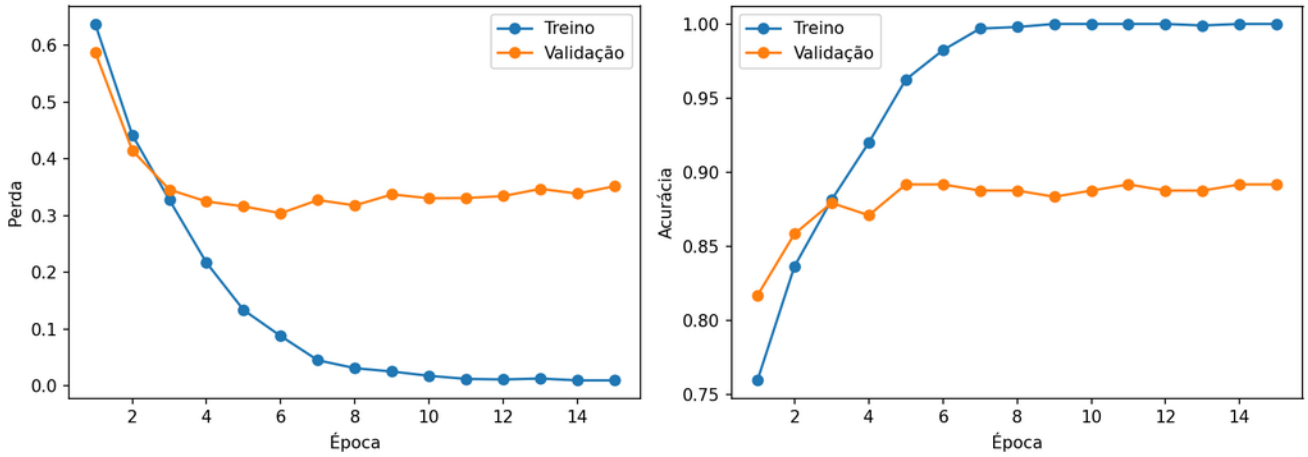
Sobre a referência do enunciado

- O enunciado atribui a um estudo da Scientific Reports a observação de que modelos com accuracy alta tinham recall abaixo de 60% na classe COVID-19.
- O artigo que abre no endereço citado é "Automated COVID-19 detection with convolutional neural networks", de Dumakude e Ezugwu (2023). Ele classifica tomografias em COVID-19, pneumonia e normal, trata o desbalanceamento com upsampling e reporta precision, recall, specificity, accuracy e F1. Eu não encontrei nele a observação sobre recall abaixo de 60%, então ela não é usada aqui como dado.

Diagnóstico: problemas técnicos e impacto clínico

A reprodução do projeto serve de evidência. Os números dela, todos da mesma execução:

Medida	Valor
Acurácia de treino na última época	100%
Acurácia de validação na última época	89,2% (214 de 240)
Acurácia de quem sempre responde Normal, na mesma validação	72,5% (174 de 240)
Recall de COVID-19 na validação do projeto	0,41 (9 de 22)
Recall de COVID-19 no teste real	0,44 (133 de 300)
Recall de Normal e de Pneumonia no teste real	0,97 e 0,94



Projeto reproduzido: perda e acurácia por época

1. Desbalanceamento sem tratamento

O que está errado

- Normal tem 70% das imagens e COVID-19 tem 10%. São 7 imagens de Normal para cada imagem de COVID-19.
- A perda dá o mesmo peso a cada imagem. Errar todas as imagens de COVID-19 custa pouco na média, então o modelo aprende a preferir Normal.
- No teste real da reprodução, 154 das 300 imagens de COVID-19 foram classificadas como Normal.

Impacto clínico

- O erro mais comum é o pior para uma triagem: um caso positivo recebe o rótulo Normal e sai da fila de prioridade.
- O paciente espera mais pelo laudo e pode circular sem isolamento nesse período.

2. Divisão sem estratificação e validação pequena

O que está errado

- Sem estratificação, o número de imagens de COVID-19 na validação depende do sorteio. Na reprodução caíram 22. Com estratificação seriam 24.

- Com 22 imagens, cada imagem vale 4,5 pontos percentuais de recall. Uma diferença de duas ou três imagens muda a conclusão.

Impacto clínico

- O hospital pode receber um modelo aprovado por duas ou três imagens de sorte. Na fila real, ele deixa passar mais casos de COVID-19 do que o número apresentado indicava, e a equipe só percebe depois, pelos laudos.

3. Acurácia global como única métrica

O que está errado

- A acurácia mistura as três classes e é dominada pela maior.
- Na reprodução, a acurácia de validação foi 89,2% e parece aceitável. Responder sempre Normal já daria 72,5%.
- Na mesma validação, a recall de COVID-19 foi 0,41. O modelo perdeu mais da metade dos casos da classe que motivou o projeto.

Impacto clínico

- É a queixa do time clínico: o modelo "ignora casos positivos". A métrica escolhida não podia mostrar o problema.

4. Sem pré-treinamento em um conjunto pequeno

O que está errado

- A ResNet-18 tem cerca de 11 milhões de parâmetros. O treino tinha 960 imagens, 98 delas de COVID-19.
- Com pesos aleatórios, a rede precisa aprender bordas e texturas a partir dessas imagens. Ela decora o treino: 100% de acurácia na época 9 da reprodução.
- Pesos da ImageNet dão filtros iniciais prontos. Radiografia é um domínio distante, então o caso pede fine-tuning com taxa de aprendizado baixa. Aqui todas as camadas foram treinadas com taxa de aprendizado baixa, e treinar só os últimos blocos, como a regra da Atividade 3 indica para dados pequenos e distantes, é a alternativa a testar.

Impacto clínico

- O impacto é o descrito no problema 5: os dois vêm de um modelo que decorou o treino.

5. Overfitting visível, taxa de aprendizado fixa e sem critério de parada

O que está errado

- O time reportou 93% no treino e 61% na validação: 32 pontos de diferença. O número já dizia que o modelo não generalizava.
- A reprodução mostra o mesmo padrão em escala menor. A perda de validação teve o mínimo na época 6 (0,304) e subiu até 0,352 na época 15, enquanto a de treino caiu para 0,009.
- SGD sem momentum, com taxa de aprendizado fixa de 0,01 e número fixo de épocas. Sem scheduler, sem weight decay, sem early stopping e sem escolha de checkpoint pela validação. O modelo entregue foi o da última época.

Impacto clínico

- O modelo decorou 98 imagens de COVID-19 e foi entregue depois do melhor ponto. Um paciente novo, de outro equipamento ou com outro posicionamento, tem a radiografia marcada como Normal e vai para o fim da fila de laudos. O time clínico só descobre quando os casos perdidos aparecem.

6. Sem augmentation

O que está errado

- Radiografias variam em posição do paciente, rotação leve, contraste e exposição. Sem augmentation, o modelo vê cada imagem sempre igual.
- Com menos de 100 imagens de COVID-19, isso facilita decorar detalhes sem relação com a doença, como marcações na borda da imagem.

Impacto clínico

- A taxa de casos perdidos passa a variar com o equipamento e com o posicionamento do paciente.

7. Sem conjunto de teste independente

O que está errado

- O mesmo conjunto de validação serviu para acompanhar o treino e para reportar o resultado. Não havia teste separado.
- Na reprodução, a acurácia no teste real, que tem 300 imagens por classe, foi 78,6%, abaixo da de validação. A recall de COVID-19 foi parecida nos dois. A acurácia caiu porque o teste tem a mesma quantidade de cada classe e a validação é dominada por Normal.

Impacto clínico

- A proporção de casos positivos em um hospital muda com a fase da epidemia. Um número medido em uma proporção fixa não informa o que acontece em outra.

8. Cópias de imagens e pacientes repetidos

O que está errado

- O projeto não descreve verificação de cópias nem divisão por paciente.
- Na base pública usada aqui havia 54 arquivos com conteúdo idêntico a outro.
- Um paciente internado faz várias radiografias. Se imagens do mesmo paciente caem no treino e na validação, o modelo é avaliado em alguém que já viu.

Impacto clínico

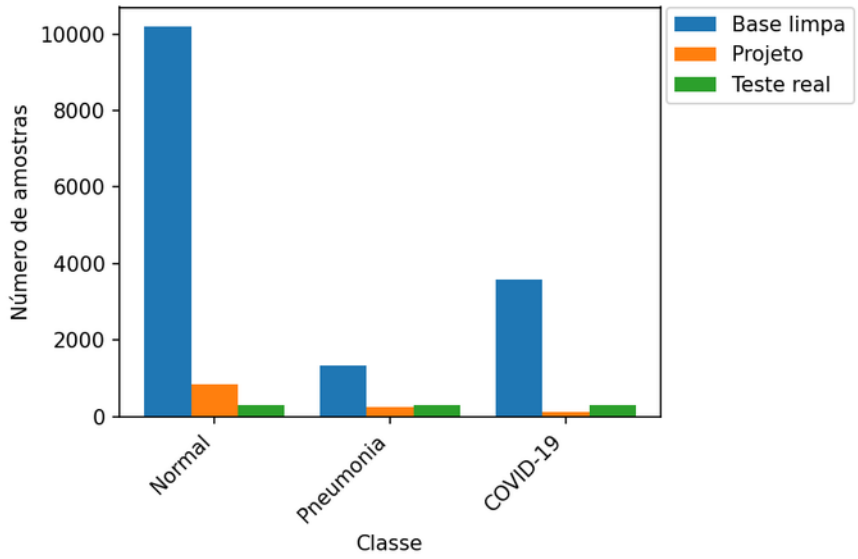
- Um paciente internado tem radiografias no treino e na validação. O modelo acerta esse paciente e a recall reportada sobe. Com pacientes novos na fila, mais casos saem como Normal do que a equipe espera, e ela confia em um resultado negativo que não merece essa confiança.

A base de dados

- Fonte: COVID-19 Radiography Database (`tawsifurrahman/covid19-radiography-database`), pública no Kaggle.

- Entraram só as radiografias das classes Normal, Pneumonia viral e COVID-19, em tons de cinza e reduzidas para 64x64. As máscaras de pulmão que acompanham a base nunca são lidas.
- 15.153 arquivos de radiografia. 54 cópias exatas removidas pelo conteúdo do arquivo. Sobram 15.099 imagens distintas.

Classe	Base limpa	Conjunto do projeto	Treino	Validação	Teste real
Normal	10.191	840	672	168	300
Pneumonia	1.338	240	192	48	300
COVID-19	3.570	120	96	24	300
Total	15.099	1.200	960	240	900



Número de imagens por classe na base limpa, no conjunto do projeto e no teste real

- **Conjunto do projeto:** 1.200 imagens sorteadas com a seed 42, nas quantidades do projeto que falhou.
- **Teste real:** 300 imagens por classe, sorteadas entre as que sobraram. Nenhuma imagem está nos dois conjuntos. A base tem milhares de imagens a mais que o projeto, então a recall pode ser medida em 300 imagens de COVID-19. A validação tem 24.
- **Treino e validação:** divisão 80/20 estratificada do conjunto do projeto. A reprodução do projeto original usa a divisão dele, sem estratificação.
- **Controles de vazamento:** a GAN só vê o treino. Validação e teste nunca recebem imagem gerada. Todos os modelos são medidos no mesmo teste real.
- Limite: só cópias exatas foram removidas, e os arquivos não trazem identificador de paciente. O problema 8 do diagnóstico vale também para este estudo.

GAN condicional para a classe COVID-19

Por que GAN condicional

- O objetivo é gerar imagens de uma classe específica. Com o rótulo como entrada, uma única rede aprende as três classes e usa as 864 imagens de Normal e Pneumonia para aprender a anatomia do tórax, que as 96 de COVID-19 sozinhas não ensinam.
- Uma CycleGAN tem dois generators e dois discriminators e custa mais tempo na T4. Ficou fora.

Generator

- Entrada: ruído de 100 dimensões e o rótulo da classe.
- Condicionamento: o rótulo passa por um embedding de 100 dimensões, concatenado ao ruído. A rede sabe a classe desde a primeira camada.
- Cinco convoluções transpostas levam de 1x1 a 64x64, com 256, 128, 64 e 32 canais, batch normalization e ReLU. A saída tem 1 canal e tanh, na faixa de -1 a 1.

Discriminator

- Entrada: a imagem de 64x64 e o rótulo.
- Condicionamento: o rótulo vira um mapa aprendido de 64x64, empilhado na imagem como segundo canal. O discriminator julga se a imagem é real para aquela classe.
- Quatro convoluções com stride 2 levam de 64x64 a 4x4, com 32, 64, 128 e 256 canais, batch normalization a partir da segunda convolução e LeakyReLU. Uma convolução final produz um logit.

O loop adversarial, a cada batch

1. Passo do discriminator: recebe as imagens reais do batch com seus rótulos e imagens geradas com os mesmos rótulos. As geradas são produzidas sem gradiente, então esse passo não altera o generator. A perda é a entropia cruzada binária (BCE): alvo 1 para as reais e 0 para as geradas.
2. Passo do generator: sorteia ruído novo, gera imagens e passa pelo discriminator já atualizado, com os pesos dele congelados. A perda é a BCE com alvo 1, a forma non-saturating: o generator ganha quando o discriminator chama as geradas de reais.
3. Cada rede tem o seu otimizador Adam, com taxa de aprendizado 0,0002 e betas (0,5, 0,999). Um passo de cada rede por batch.

- Treino: 286 épocas, batch de 64, 15 batches por época, 0,67 segundo por época na T4.
- O número de épocas veio do limite de tempo do notebook, que rodou em 23,0 minutos com pico de 656,5 MB de GPU.

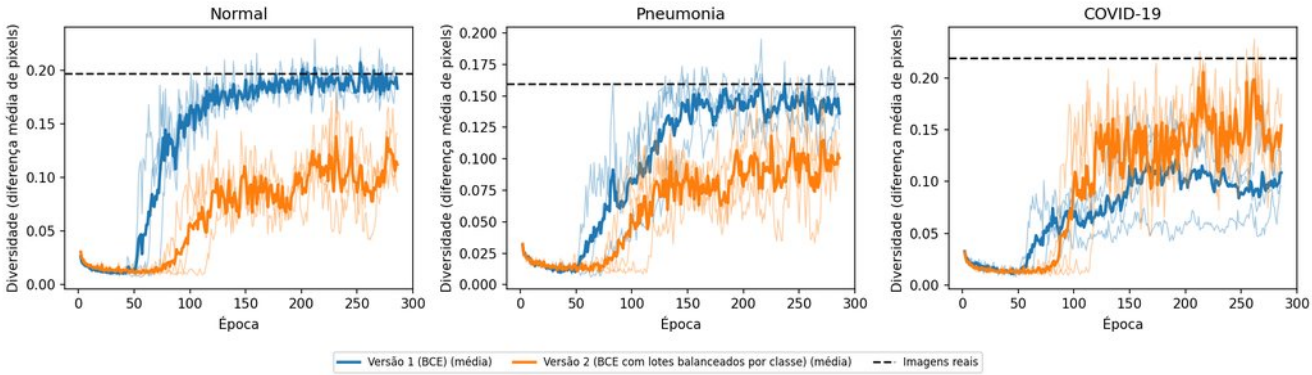
Instabilidade do treinamento e mitigação

Como a instabilidade foi medida

- **Diversidade:** diferença média de pixels entre pares de imagens geradas da mesma classe, com ruído fixo, a cada época. Perto de zero quer dizer que o generator produz quase a mesma imagem para qualquer ruído: mode collapse.
- **Razão:** diversidade gerada nas últimas 29 épocas dividida pela diversidade das imagens reais de treino da classe (0,197 em Normal, 0,160 em Pneumonia, 0,219 em COVID-19). Razão abaixo de 0,5 marca a classe como colapsada.
- **Divergência:** perda não finita, ou perda do discriminator abaixo de 0,2 no fim, sinal de que ele venceu e o generator ficou sem gradiente útil.
- A época 1 não é interpretada. Nela a medida de diversidade não é confiável.

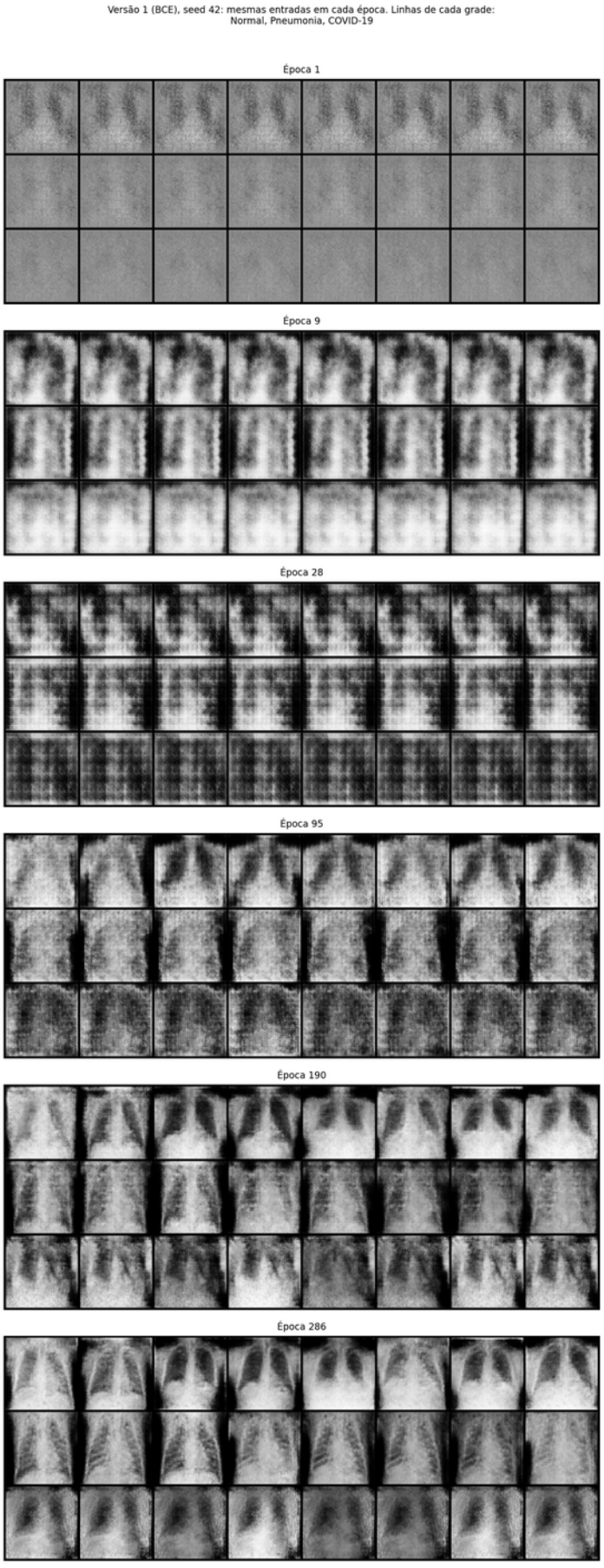
Diagnóstico da versão 1

A versão 1 é o baseline: a rede e o loop descritos acima, com os batches sorteados como vêm dos dados.

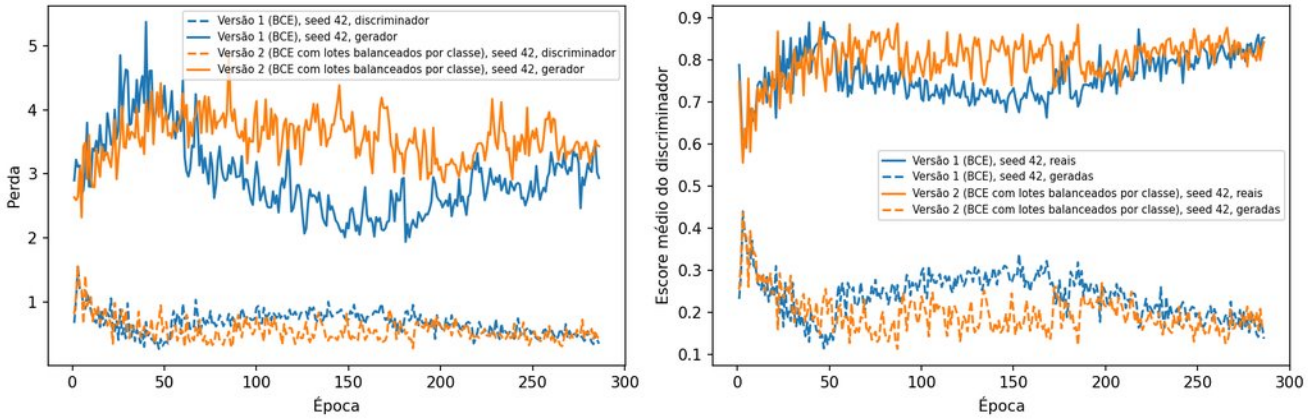


Diversidade por época, um painel por classe, nas duas versões. Linha grossa: média das três seeds. Linhas finas: cada seed. Linha tracejada: diversidade das imagens reais

- **Mode collapse no começo, em todas as seeds.** A diversidade fica entre 0,01 e 0,03 até cerca da época 50 e só passa de 0,05 entre as épocas 53 e 101, conforme a seed e a classe. Na grade de imagens, as oito imagens de cada classe são iguais nas épocas 9 e 28.
- **Saída do colapso desigual entre classes.** Normal sobe perto da época 50 e chega ao nível das imagens reais. Pneumonia e COVID-19 sobem depois e param abaixo da linha das reais, COVID-19 mais longe.
- **Sem divergência.** Nenhuma perda não finita. Na seed 42, a perda do discriminator termina em 0,47 e a do generator em 3,10. No gráfico, o escore médio fica perto de 0,82 para imagens reais e 0,18 para geradas no fim do treino.
- **A classe minoritária é a menos diversa.** A razão média de COVID-19 foi 0,439, contra 0,956 em Normal e 0,890 em Pneumonia.
- **Colapso de COVID-19 em duas de três seeds.** As razões foram 0,412 (seed 42), 0,602 (seed 43) e 0,303 (seed 44), com o limite em 0,5. Na seed 42 a curva chega a cerca de 0,17 perto da época 155 e termina perto de 0,09.



Versão 1, seed 42: imagens geradas com o mesmo ruído em seis épocas



Perdas e escores do discriminator, seed 42, nas duas versões

Causa provável

- As imagens falsas de cada batch recebem os rótulos do batch real. COVID-19 tem 96 das 960 imagens, então só cerca de 10% das atualizações do generator são dessa classe.

Quatro mitigações tentadas

Três foram testadas em execuções anteriores do notebook e saíram dele. Os números são dessas execuções, guardados no arquivo de resultados como `earlier_executions`. São razões de diversidade em COVID-19, com uma seed por versão. Nas três execuções, o baseline BCE teve 0,66, 0,56 e 0,65.

1. **Spectral normalization com hinge loss e taxas de aprendizado diferentes para as duas redes: falhou.** COVID-19 ficou colapsada nas três execuções: 0,04, 0,04 e 0,18.
2. **Augmentation diferenciável (translação e brilho aleatórios antes do discriminator): falhou.** Testada na execução 3, teve 0,45 e ficou abaixo do baseline nas três classes.
3. **WGAN-GP: falhou.** Com uma seed parecia a melhor versão: 0,76 e 0,91 nas execuções 2 e 3. Em uma execução anterior com três seeds e 240 épocas, a razão de COVID-19 foi 0,485 ± 0,370, contra 0,591 ± 0,051 no baseline. Das três seeds, a 43 terminou colapsada (razão 0,061).
4. **Batches balanceados por classe: melhora parcial.** É a versão 2 do notebook, descrita abaixo.

Por que estabilizadores padrão podem falhar aqui

- Hipótese, não testada em separado: o treino é curto para essas técnicas. São 960 imagens e 15 batches por época, 4.290 passos em 286 épocas. Elas foram propostas para datasets com dezenas de milhares de imagens e treinos muito mais longos. Com poucos passos, o sinal que chega ao generator pode ficar fraco, e ele não sai do colapso inicial.
- Os valores mudam de uma execução para outra. O baseline com 32 canais teve 0,56 na execução 2 e 0,65 na execução 3, com uma seed em cada. Uma hipótese é que a GPU não repete o treino de uma GAN bit a bit e que pequenas diferenças no começo crescem. O arquivo de resultados não registra se as duas execuções tinham as mesmas configurações. Por isso a comparação final usa três seeds e olha média e desvio padrão.

Versão 2: batches balanceados por classe

- Mesma rede, mesma perda e mesmas taxas de aprendizado da versão 1. Muda só o sorteio: cada imagem é sorteada com reposição, com probabilidade inversa à frequência da sua classe. Cada classe passa a ter um terço esperado de cada batch.
- O número de batches por época fica igual. A mudança ataca a causa apontada no diagnóstico.

Razão de diversidade, três seeds (média ± desvio padrão amostral)

Classe	Versão 1	Seeds colapsadas	Versão 2	Seeds colapsadas
Normal	0,956 ± 0,032	0 de 3	0,522 ± 0,086	1 de 3
Pneumonia	0,890 ± 0,092	0 de 3	0,588 ± 0,094	1 de 3
COVID-19	0,439 ± 0,151	2 de 3	0,700 ± 0,043	0 de 3

- COVID-19 por seed: versão 1 com 0,412, 0,602 e 0,303. Versão 2 com 0,698, 0,744 e 0,658.

O que melhorou

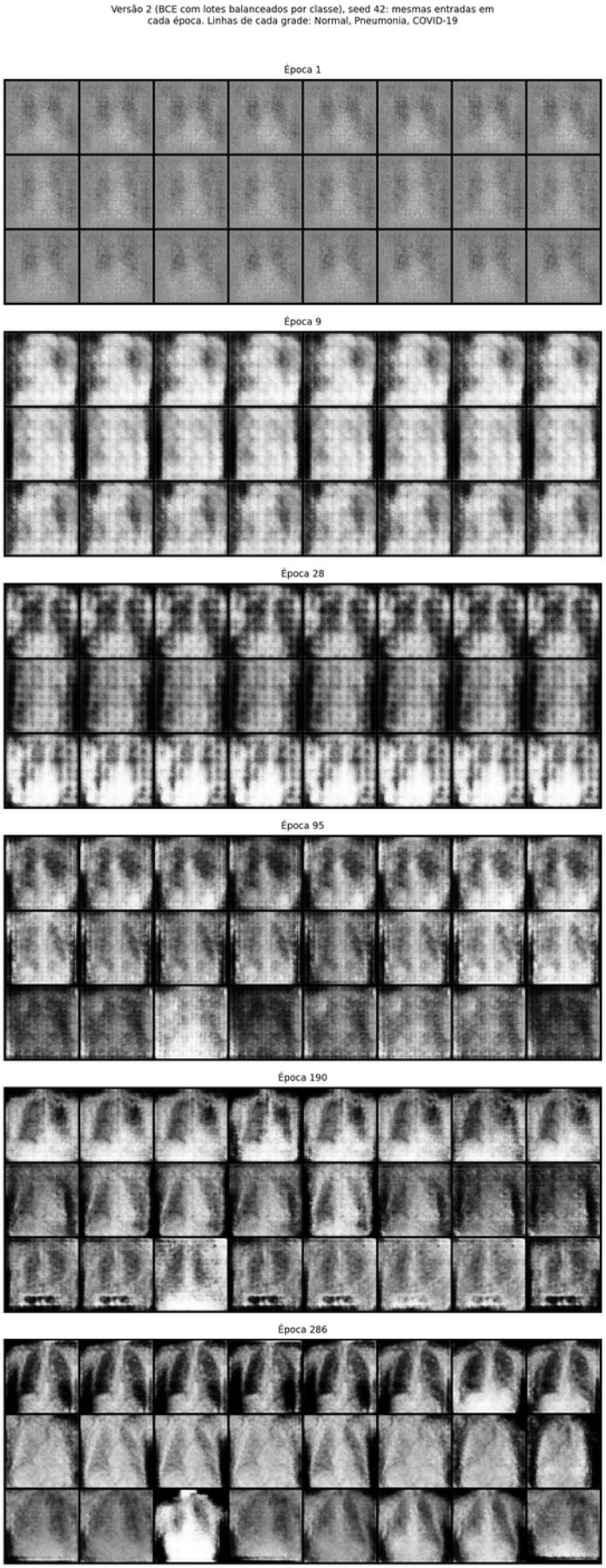
- Nesta execução, a diversidade média de COVID-19 subiu 0,261, de 0,439 para 0,700. As três seeds da versão 2 (0,658 a 0,744) ficaram acima das três da versão 1 (0,303 a 0,602).
- A estabilidade da classe minoritária entre seeds. O desvio padrão de COVID-19 caiu de 0,151 para 0,043, cerca de 3,5 vezes menor.
- Nenhuma seed da versão 2 ficou colapsada em COVID-19. Na versão 1, duas de três ficaram (0,412 e 0,303).

O que não foi mostrado

- Que a melhora venha só do balanceamento, nem o tamanho dela. A execução anterior deste notebook, com as mesmas configurações e as mesmas seeds, deu 0,577 ± 0,106 na versão 1 e 0,660 ± 0,021 na versão 2: um ganho de 0,083, menor que o desvio padrão da versão 1. Nesta execução o ganho foi 0,261. A versão 2 mudou pouco entre as duas execuções (0,660 e 0,700). A versão 1 mudou muito (0,577 e 0,439) e ficou abaixo das três execuções anteriores do baseline com uma seed (0,66, 0,56 e 0,65). Parte da distância desta execução vem da variação entre execuções da GAN.
- O que se repetiu nas duas execuções: nenhuma seed da versão 2 colapsou em COVID-19 (0 de 6), contra 3 de 6 na versão 1, e o desvio padrão da versão 2 foi o menor. Os números da execução anterior não estão no arquivo de resultados desta execução.

O que custou

- Normal e Pneumonia perderam diversidade, como mostra a tabela. Cada imagem dessas classes passou a aparecer em menos atualizações.
- No gráfico, a versão 2 sai do colapso mais tarde: entre as épocas 84 e 119, contra 53 a 101 na versão 1. Na seed 43, as três classes ficam colapsadas até perto da época 118.
- Nenhuma das duas versões diverge. As curvas de perda da seed 42 são parecidas.



Versão 2, seed 42: imagens geradas com o mesmo ruído em seis épocas

Versão escolhida para o experimento de recall

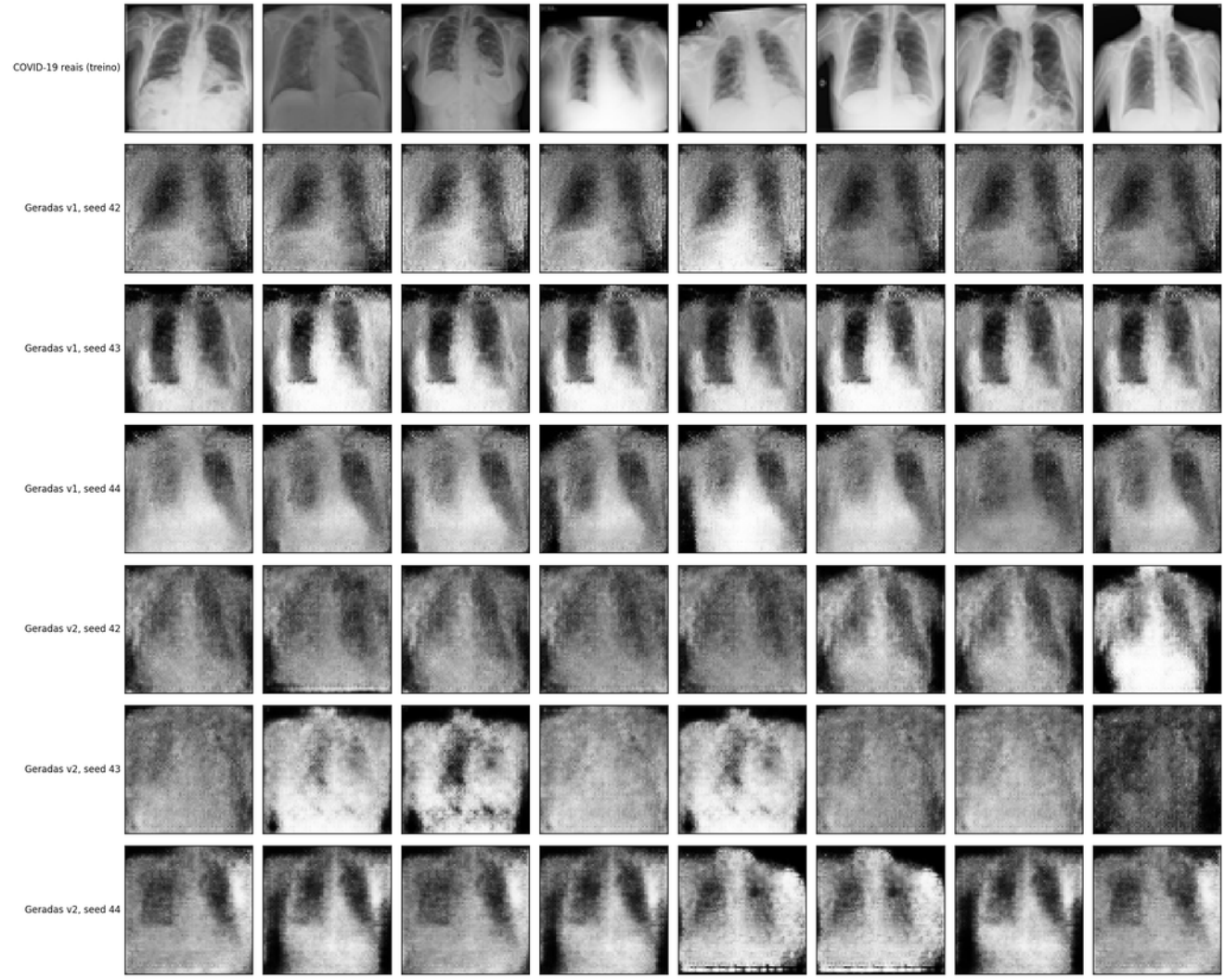
- Regra fixada antes do experimento: a versão com a maior razão média de diversidade em COVID-19 nas três seeds. A regra escolheu a versão 2.

As imagens geradas são cópias?

- Uma GAN treinada com 96 imagens pode decorar essas imagens, e cópias não acrescentam nada ao classificador.
- Medida: para cada imagem gerada de COVID-19, a diferença média de pixels até a imagem real de COVID-19 mais próxima no treino. A régua é a mesma conta feita com as 24 imagens reais de COVID-19 da validação.

Distância até a imagem de treino mais próxima	Imagens	Média	Mediana	Mínimo
Geradas pela versão 2 (três seeds)	1.728	0,119	0,119	0,077
Reais de validação	24	0,105	0,099	0,072

- Nenhuma imagem gerada ficou mais perto do treino que a imagem real de validação mais próxima. A parcela foi 0. Não há sinal de cópia.
- As geradas ficam um pouco mais longe do treino que as reais de validação. Parte dessa distância é ruído.
- Segundo controle: balancear poderia aumentar a diversidade só por gerar ruído. A distância média por seed foi 0,112, 0,128 e 0,107 na versão 1 e 0,125, 0,098 e 0,134 na versão 2. As médias são 0,116 e 0,119: a diferença é menor que a variação entre seeds.



Primeira linha: COVID-19 reais do treino. Demais linhas: imagens geradas por cada versão e seed

O que se vê nas imagens

- As reais são nítidas e variam bastante em contraste, enquadramento e posição do paciente.
- As geradas mostram o contorno do tórax e dos pulmões, mas são granuladas, sem costelas nem detalhes finos. Dentro de uma mesma seed, as oito são parecidas entre si na versão 1: o mesmo tórax com pequenas mudanças de brilho. Na versão 2 há mais variação, sobretudo na seed 43.
- Limite da medida: distância de pixels em 64x64 detecta cópia exata ou quase exata. Ela não mede se a imagem gerada mostra os sinais que um radiologista procura.

Efeito das imagens sintéticas na recall

Classificador, igual nos três braços

- ResNet-18 com pesos da ImageNet, todas as camadas treinadas. Adam, taxa de aprendizado 0,0001, 10 épocas, batch de 32.
- Entrada de 64x64 ampliada para 128. Augmentation leve: translação de até 4 pixels e rotação de até 7 graus. Sem flip horizontal, porque esquerda e direita importam em uma radiografia de tórax.

Os três braços

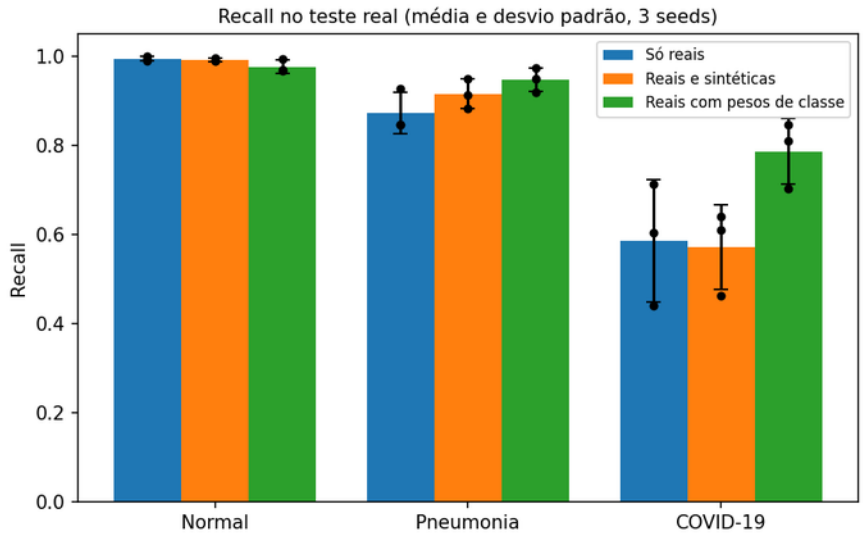
- **Braço 1, só reais:** as 960 imagens de treino.

- **Braço 2, reais e sintéticas:** as 960 mais 576 imagens geradas de COVID-19, o que iguala COVID-19 a Normal (672 cada). O classificador da seed i usa a GAN da seed i.
- **Braço 3, reais com pesos de classe:** as 960 imagens, com a perda ponderada pelo inverso da frequência de cada classe. É a alternativa sem GAN.
- Três seeds por braço (42, 43, 44). Medida no teste real, 300 imagens por classe.

Recall no teste real (média ± desvio padrão amostral, três seeds)

Braço	Normal	Pneumonia	COVID-19	COVID-19 por seed
1. Só reais	0,994 ± 0,005	0,873 ± 0,046	0,586 ± 0,138	0,713 / 0,440 / 0,603
2. Reais e sintéticas	0,992 ± 0,004	0,916 ± 0,033	0,571 ± 0,095	0,640 / 0,610 / 0,463
3. Reais com pesos de classe	0,977 ± 0,015	0,948 ± 0,027	0,787 ± 0,074	0,810 / 0,703 / 0,847

Nesta execução, o braço com pesos de classe fica acima dos outros dois por mais que o desvio padrão de cada um. Na execução anterior, com as mesmas configurações, o braço com pesos de classe também era o mais alto, mas a diferença ficava dentro do desvio. Os braços 1 e 2 não se distinguem.



Recall por classe nos três braços, com o desvio padrão entre seeds

- Braço 2 menos braço 1: -0,014. O desvio padrão do braço 1 é 0,138.
- Braço 3 menos braço 1: 0,201, acima de 0,138.
- Braço 3 menos braço 2: 0,216. Os desvios padrão dos braços 2 e 3 são 0,095 e 0,074.
- Execução anterior, mesmas seeds: 0,583 ± 0,160, 0,663 ± 0,044 e 0,736 ± 0,102. O braço 3 menos o braço 1 dava 0,153, abaixo do desvio padrão do braço 1. O braço 2 ficava 0,080 acima do braço 1 e nesta execução ficou 0,014 abaixo: o sinal trocou.
- Com três seeds a separação não é total: a pior seed do braço 3 (0,703) ficou abaixo da melhor seed do braço 1 (0,713).
- Nesta execução, o braço 3 teve o menor desvio padrão (0,074). Na anterior foi o braço 2.

Confusão que não foi controlada

- O braço 2 treina com 1.536 imagens contra 960 nos outros dois. Nas mesmas 10 épocas, são 1,6 vez mais atualizações de pesos.
- Qualquer diferença do braço 2, a favor ou contra, poderia vir do treino mais longo. O experimento não separa as duas coisas.

Para onde vão os casos perdidos (900 imagens de COVID-19: 300 por seed)

Braço	Acertos	Classificadas como Normal	Classificadas como Pneumonia
1. Só reais	527	356	17
2. Reais e sintéticas	514	376	10
3. Reais com pesos de classe	708	176	16

- Quase todo caso perdido de COVID-19 vira Normal, nos três braços.
- A precision de COVID-19 fica entre 0,98 e 0,99 nos três braços. Quando o modelo diz COVID-19, quase sempre acerta. Ele diz pouco.
- A precision de Normal é 0,661, 0,667 e 0,802 no teste balanceado. Um resultado Normal desses modelos não é confiável como resultado negativo.
- Mesmo o melhor braço perde em média 21% dos casos de COVID-19 do teste.

Conclusão para o plano

- Pesos de classe custam uma linha de código e nenhum treino extra. São a primeira coisa a tentar, junto com um protocolo correto.
- Este estudo não mostrou que a augmentation sintética aumenta a recall: a do braço 2 (0,571) ficou 0,014 abaixo da do braço 1 (0,586), diferença muito menor que a variação entre seeds. O braço com pesos de classe teve a maior recall de COVID-19 nas duas execuções (0,736 e 0,787).

Plano de melhoria integrado

Problema do diagnóstico	Ação
1. Desbalanceamento	Pesos de classe na perda como primeira medida. Coletar mais casos reais de COVID-19, que é a correção de fato.
2. Divisão sem estratificação	Divisão estratificada. Validação cruzada com pelo menos 5 partes, para reportar média e intervalo.
3. Só acurácia	Métrica principal: recall de COVID-19 em uma specificity fixa. Reportar também precision, F1 e matriz de confusão por classe, com intervalo de confiança.
4. Sem pré-treinamento	ResNet-18 pré-treinada, com fine-tuning e taxa de aprendizado baixa. Comparar com pesos pré-treinados em radiografias, se houver.
5. Overfitting e otimização	Adam ou SGD com momentum, scheduler de taxa de aprendizado, weight decay, early stopping e escolha de checkpoint pela recall de validação.
6. Sem augmentation	Translação, rotação leve, brilho e contraste. Sem flip horizontal.
7. Sem teste	Teste separado desde o início, usado uma única vez, e o teste externo descrito abaixo.
8. Cópias e pacientes	Remover cópias exatas e quase-duplicatas. Dividir por identificador de paciente.

Modelo

- Entrada em resolução maior que 64x64. É provável que achados sutis de COVID-19 em radiografia se percam nessa redução.
- Limiar de decisão ajustado na validação para atingir a recall pedida.

Augmentation sintética

- Entra só depois das ações acima, como experimento, com os controles de vazamento e a verificação de cópias deste estudo.
- Comparação controlada: o braço com sintéticas contra um braço de imagens reais com o mesmo número de atualizações e contra o braço com pesos de classe, com pelo menos 5 seeds.
- Critério para adotar: recall de COVID-19 maior que a do braço com pesos de classe, com diferença acima da variação entre seeds, no teste externo.

Critério para adoção clínica

Os valores abaixo são propostas, a discutir e fechar com o time clínico.

1. **Teste externo.** Radiografias de outro hospital, com outros equipamentos, separadas por paciente e nunca usadas no desenvolvimento.
2. **Recall mínima em specificity fixa.** Proposta: recall de COVID-19 de pelo menos 0,90, com o limite inferior do intervalo de confiança de 95% acima de 0,85, em specificity de pelo menos 0,80. O time clínico define os valores conforme o custo de um caso perdido e a carga de revisão que aguenta.
3. **Tamanho do teste.** Casos positivos suficientes para o intervalo ser estreito. Com 300 positivos e recall perto de 0,90, o intervalo de 95% tem cerca de 0,03 para cada lado.
4. **Estabilidade.** A pior seed e o pior subgrupo (equipamento, projeção, faixa etária) também acima do mínimo.
5. **Validação prospectiva.** Um período em que o sistema roda em silêncio, sem influenciar o atendimento, e a saída é comparada com o laudo e com o teste laboratorial.
6. **Radiologista no circuito.** O sistema ordena a fila de laudos. Ele não libera nenhum paciente. Todo exame é lido por um radiologista, e os resultados negativos com incerteza acima de um limiar combinado recebem revisão prioritária.
7. **Monitoramento depois da implantação.** Auditoria mensal de uma amostra, com recall e número de casos perdidos, acompanhamento da proporção de positivos e de mudanças de equipamento, e uma regra escrita para suspender o sistema se a recall cair abaixo do mínimo.

Limites deste estudo

- **Este estudo não tem validade clínica.** É um exercício de método com imagens públicas.
- **Resolução.** 64x64 pixels é muito menos do que um radiologista usa. A escolha veio do tempo de treino da GAN na T4.
- **Uma base montada de várias fontes.** As classes da base vêm de coleções diferentes. Um classificador pode aprender a reconhecer a origem da imagem e usar isso como atalho. Esse efeito não foi medido, e o teste vem da mesma base.
- **Três seeds.** Bastam para mostrar que a variação é grande. Não bastam para ordenar os braços 1 e 2, nem para medir com precisão a vantagem do braço 3 e a das versões da GAN.
- **Um sorteio do conjunto do projeto.** Outro sorteio daria outras 96 imagens de COVID-19 no treino.
- **Teste balanceado.** O teste tem 300 imagens por classe. A acurácia e a precision medidas nele não valem para a proporção de casos de um hospital. A recall por classe não depende dessa proporção.
- **Rótulos.** São os da base, sem revisão por radiologista.

Atividade 4.2: Transfer learning para análise de tráfego urbano

O projeto analisado

- Tarefa: classificar o fluxo visto por câmeras urbanas como "livre", "moderado" ou "congestionado".
- Modelo: ResNet-50 pré-treinada na ImageNet, com fine-tuning.
- Dados: 800 frames rotulados de 12 câmeras.
- Resultado: 78% de acurácia na validação. O sistema foi implantado.
- Em produção: falhas sistemáticas em chuva, à noite e em câmeras com ângulos que não estavam no treino.
- Esta seção é só análise. Nada foi implementado.
- A página indicada no enunciado (Meegle, "Transfer Learning For Traffic Analysis") lista quatro desafios de transfer learning nessa área: diferença entre o domínio de origem e o de destino, overfitting por fine-tuning agressivo, custo computacional e dificuldade de interpretar o modelo. Os dois primeiros são plausíveis neste projeto.

Problemas técnicos e metodológicos

1. A validação pode usar as mesmas câmeras do treino

- **Problema.** Provavelmente os frames de validação vêm das 12 câmeras do treino. Cada câmera tem fundo fixo: os mesmos prédios, faixas e postes. O modelo pode reconhecer a câmera e o seu padrão habitual. Nesse caso, os 78% medem frames novos de câmeras conhecidas e não informam nada sobre uma câmera nova.
- **Impacto operacional.** O sistema foi instalado em câmeras com outros ângulos. Se a validação usou as câmeras do treino, a acurácia nas câmeras novas nunca foi medida.
- **Como abordar.** Dividir por câmera: todas as imagens de uma câmera ficam no mesmo conjunto. Com 12 câmeras, usar validação cruzada deixando uma câmera de fora por vez. É a mesma lógica da divisão por pintor da Atividade 1, que mede o modelo em um pintor nunca visto.

2. Domain shift em chuva e à noite

- **Problema.** O treino não cobria essas condições, ou cobria pouco. Na chuva há reflexo no asfalto, gotas na lente e contraste baixo. À noite os veículos viram pares de faróis sobre fundo escuro, com ruído do sensor. A distribuição da entrada muda, e o modelo foi ajustado para outra.
- **Impacto operacional.** Chuva e pico noturno são momentos em que o congestionamento é comum e a central mais precisa do sistema. Ele falha junto com a demanda.
- **Como abordar.** Coletar e rotular frames de chuva, noite, amanhecer e entardecer em todas as câmeras. Reportar acurácia e recall por condição. Usar augmentation ligada a essas condições: brilho e contraste baixos, ruído, desfoque, reflexos. A augmentation complementa os dados reais.

3. Poucos frames, e correlacionados no tempo

- **Problema.** São 800 frames para 12 câmeras, cerca de 67 por câmera, divididos em três classes. Se os frames foram tirados com segundos de diferença, são quase a mesma imagem, e o número de exemplos independentes é bem menor que 800. Se frames vizinhos caem no treino e na validação, a validação mede memória.

- **Impacto operacional.** A acurácia reportada fica otimista, e o fine-tuning de uma ResNet-50 com milhões de parâmetros tende a overfitting. A Atividade 3 deste relatório teve o mesmo tipo de vazamento, com imagens duplicadas.
- **Como abordar.** Amostrar frames espaçados no tempo, em dias e horários diferentes. Congelar a maior parte do backbone enquanto houver poucos dados.

4. Features de objeto para uma propriedade da cena

- **Problema.** A ImageNet ensina a reconhecer objetos. Fluxo é quantos veículos ocupam a via e a que velocidade andam. O classificador de imagem inteira não conta veículos nem mede ocupação de forma explícita. Ele pode aprender atalhos, como a hora do dia pela luz ou a câmera pelo fundo.
- **Impacto operacional.** O sistema acerta enquanto o atalho vale e erra quando ele deixa de valer: um acidente às 3 da manhã, uma via vazia em feriado no horário de pico.
- **Como abordar.** Medir a quantidade que define o fluxo. Detectar e contar veículos em uma região da via, calcular a ocupação por faixa e classificar a partir desses números. O resultado fica interpretável: o operador vê a contagem que levou ao rótulo.

5. Fronteira subjetiva entre as classes

- **Problema.** Não há definição objetiva de onde termina "moderado" e começa "congestionado". Anotadores diferentes rotulam o mesmo frame de formas diferentes. O mesmo número de veículos é fluxo livre em uma avenida de quatro faixas e congestionamento em uma rua de uma faixa. Parte dos erros na fronteira é ruído de rótulo.
- **Impacto operacional.** Os alertas oscilam entre "moderado" e "congestionado" em situações estáveis. O operador deixa de confiar nos alertas.
- **Como abordar.** Definir as classes por medidas: ocupação da via, velocidade média ou comprimento de fila, com limiares por tipo de via. Rotular cada frame com dois anotadores e medir a concordância. Reportar a matriz de confusão, para separar o erro entre classes vizinhas do erro entre "livre" e "congestionado".

6. Acurácia global e nenhum monitoramento depois da implantação

- **Problema.** A decisão de implantar se apoiou em um número só. O projeto não informa a proporção das classes nem a recall de "congestionado". Se, em produção, as falhas foram percebidas só por quem usava o sistema, faltou medição contínua.
- **Impacto operacional.** Se "livre" for a classe mais comum, um modelo pode chegar perto de 78% e ainda perder boa parte dos congestionamentos. Na Atividade 4.1, uma acurácia de validação de 89,2% convivia com recall de 0,41 na classe que importava. Sem monitoramento, a degradação por câmera reposicionada, lente suja ou obra na via só aparece como reclamação.
- **Como abordar.** Reportar recall e precision por classe, por câmera e por condição. Depois da implantação, auditar uma amostra rotulada por semana, acompanhar a distribuição das previsões e da confiança por câmera e definir um alerta para quando elas mudarem.

Riscos de partir da ImageNet para classificar fluxo

- **Objeto contra cena.** As features da ImageNet descrevem, em geral, um objeto central, grande e bem iluminado. Em uma câmera de tráfego, a resposta está na cena inteira: densidade, ocupação e comprimento de fila. Nenhum veículo isolado define a classe.
- **Ponto de vista.** As fotos da ImageNet são, em geral, tiradas na altura dos olhos e de perto. A câmera de tráfego é fixa, alta e de ângulo aberto. Os veículos aparecem pequenos, de cima e parcialmente cobertos uns pelos outros. As camadas finais do backbone foram treinadas para outra escala de objeto.

- **Frame contra estado.** Congestionamento é um estado ao longo do tempo. Um frame com muitos carros pode ser uma fila parada ou um pelotão passando em velocidade normal. Um modelo de imagem única não tem como separar os dois casos.
- **Condições.** Em geral, a ImageNet é formada sobretudo por fotos bem iluminadas, e o fine-tuning com 800 frames não cobre a diferença para chuva, noite e ofuscamento.
- **O que ainda se aproveita.** As primeiras camadas (bordas, texturas) servem em qualquer domínio. O risco está em confiar nas camadas finais e em um conjunto pequeno para adaptá-las. Pela regra da Atividade 3, este é o caso de dataset pequeno e domínio distante, o mais difícil.

Como eu conduziria o projeto

1. **Definir a tarefa com a central de tráfego.** Classes definidas por ocupação, velocidade ou fila, com limiares por tipo de via. Decidir qual erro custa mais: perder um congestionamento ou gerar um alerta falso.
2. **Coletar dados com cobertura planejada.** Mais câmeras e mais ângulos. Frames espaçados no tempo, em dias diferentes, com chuva, noite, amanhecer e entardecer. Dois anotadores por frame.
3. **Dividir por câmera e por dia.** Validação cruzada por câmera, e um conjunto de teste com câmeras e dias nunca vistos, usado uma única vez.
4. **Começar por um baseline que mede o fluxo.** Um detector de veículos pré-treinado, contagem e ocupação por região da via, e limiares simples. Em seguida, um baseline temporal: a mesma medida em uma janela de frames, ou a diferença entre frames para estimar movimento.
5. **Só depois, o classificador de imagem.** Fine-tuning dos últimos blocos, com taxa de aprendizado baixa e augmentation ligada às condições reais: brilho, contraste, ruído, desfoque, pequenas mudanças de escala e perspectiva. Ele entra se superar os baselines no teste por câmera.
6. **Avaliar por fatia.** Recall, precision e matriz de confusão por classe, por câmera e por condição (dia, noite, chuva), com intervalo de confiança.
7. **Implantar em etapas.** Primeiro em silêncio, ao lado da operação atual, em poucas câmeras. Para cada câmera nova, rotular uma amostra pequena e medir antes de ativar.
8. **Monitorar.** Auditoria semanal de amostras, acompanhamento da distribuição das previsões por câmera, alerta para mudanças e um ciclo para rotular os erros e treinar de novo.

Uso de IA e referências

Uso de IA

Ferramenta

- Claude, Gemini.

Para que foi usada

- Planejar o projeto: dividir o trabalho em tarefas e definir a ordem de execução.
- Escrever e revisar o código do pacote `src/dlv`, dos scripts e dos testes.
- Rodar os experimentos no Google Colab, em uma GPU T4.

Como o resultado foi conferido

- Testes automáticos. O repositório tem 394 testes, que rodam na CPU com entradas pequenas.
- Revisão independente. Cada tarefa passou por uma revisão independente antes de ser aceita.
- Números do relatório. Cada número foi comparado com os arquivos de resultado em `results/`, gerados pelas execuções dos notebooks.
- Problemas de dados achados e corrigidos no caminho:
  - Atividade 3: o download trazia as imagens duplicadas, e as cópias foram removidas antes da divisão.
  - Atividade 2: as miniaturas contavam a mesma figura duas vezes, e foram removidas.
  - Atividade 4: as máscaras de pulmão que acompanham as radiografias nunca são lidas, e o primeiro resultado da GAN não se repetia entre seeds, o que levou ao estudo de instabilidade.
- Resultados fracos foram escritos como saíram. Hiperparâmetros não foram ajustados para chegar a uma meta.

Responsabilidade

As conclusões do relatório são minhas, e os erros que restarem também.

Referências

Dados

- WikiArt, cópia `Artificio/WikiArt` no Hugging Face. Página: https://huggingface.co/datasets/Artificio/WikiArt. A página não declara licença. É um upload de terceiros, usado aqui só para estudo.
- ADS-16 Computational Advertising Dataset, de Giorgio Roffo (`groffo/ads16-dataset`). Página: https://www.kaggle.com/datasets/groffo/ads16-dataset. Licença na página: "Other (specified in description)". A descrição pede a citação de Roffo, G. e Vinciarelli, A. (2016), "Personality in computational advertising: A benchmark", 4th Workshop on Emotions and Personality in Personalized Systems (EMPIRE) 2016.
- Images Dataset, de Pavan Sanagapati (`pavansanagapati/images-dataset`). Página: https://www.kaggle.com/datasets/pavansanagapati/images-dataset. Licença na página: "CC0: Public Domain".
- COVID-19 Radiography Database, de Tawsifur Rahman (`tawsifurrahman/covid19-radiography-database`). Página: https://www.kaggle.com/datasets/tawsifurrahman/covid19-radiography-database. Licença na página: "Data files © Original Authors". A página pede a citação de dois artigos:

- M.E.H. Chowdhury, T. Rahman, A. Khandakar, R. Mazhar, M.A. Kadir, Z.B. Mahbub, K.R. Islam, M.S. Khan, A. Iqbal, N. Al-Emadi, M.B.I. Reaz, M. T. Islam, “Can AI help in screening Viral and COVID-19 pneumonia?” IEEE Access, Vol. 8, 2020, pp. 132665 - 132676. https://ieeexplore.ieee.org/document/9144185
- Rahman, T., Khandakar, A., Qiblawey, Y., Tahir, A., Kiranyaz, S., Kashem, S.B.A., Islam, M.T., Maadeed, S.A., Zughaier, S.M., Khan, M.S. and Chowdhury, M.E., 2020. Exploring the Effect of Image Enhancement Techniques on COVID-19 Detection using Chest X-ray Images. https://doi.org/10.1016/j.compbiomed.2021.104319

Modelos e pesos pré-treinados

- ResNet-50 da torchvision, pesos `IMAGENET1K_V2` (Atividade 3).
- ResNet-18 da torchvision, pesos da ImageNet (Atividade 4.1).
- ViT-B/16 da torchvision, pesos `IMAGENET1K_V1` (Atividade 1).
- CLIP, `openai/clip-vit-base-patch32`, no Hugging Face (Atividade 2).
- Tokenizer do `bert-base-uncased`, no Hugging Face (Atividade 2).

Bibliotecas

- PyTorch 2.11.0.
- torchvision 0.26.0.
- Hugging Face Transformers 5.18.0.
- scikit-learn 1.9.1.
- Matplotlib 3.11.2.

Os resultados do relatório foram produzidos no Google Colab, com as versões instaladas lá na época das execuções. O `uv.lock` fixa as versões usadas nos testes locais.

Artigos dos métodos

- Vaswani, A. et al. Attention Is All You Need. NeurIPS, 2017.
- Dosovitskiy, A. et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR, 2021.
- Touvron, H. et al. Training data-efficient image transformers and distillation through attention. ICML, 2021.
- Liu, Z. et al. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. ICCV, 2021.
- Bao, H. et al. BEiT: BERT Pre-Training of Image Transformers. ICLR, 2022.
- He, K. et al. Masked Autoencoders Are Scalable Vision Learners. CVPR, 2022.
- Devlin, J. et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL, 2019.
- Radford, A. et al. Learning Transferable Visual Models From Natural Language Supervision. ICML, 2021.
- He, K. et al. Deep Residual Learning for Image Recognition. CVPR, 2016.
- Goodfellow, I. et al. Generative Adversarial Nets. NeurIPS, 2014.
- Mirza, M. e Osindero, S. Conditional Generative Adversarial Nets. arXiv, 2014.
- Miyato, T. et al. Spectral Normalization for Generative Adversarial Networks. ICLR, 2018.
- Gulrajani, I. et al. Improved Training of Wasserstein GANs. NeurIPS, 2017.
- Zhao, S. et al. Differentiable Augmentation for Data-Efficient GAN Training. NeurIPS, 2020.
- Zhu, J.-Y., Park, T., Isola, P. e Efros, A. A. Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks. ICCV, 2017.
- Tan, M. e Le, Q. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML, 2019.
- Kingma, D. P. e Ba, J. Adam: A Method for Stochastic Optimization. ICLR, 2015.
- Loshchilov, I. e Hutter, F. Decoupled Weight Decay Regularization. ICLR, 2019.

- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. e Fei-Fei, L. ImageNet: A Large-Scale Hierarchical Image Database. CVPR, 2009.

Referências do enunciado

- Meegle, "Transfer Learning For Traffic Analysis", como indicada no enunciado: https://www.meegle.com/en_us/topics/transfer-learning/transfer-learning-for-traffic-analysis
- Dumakude e Ezugwu. Automated COVID-19 detection with convolutional neural networks. Scientific Reports, 2023. https://www.nature.com/articles/s41598-023-37743-4